Group Project

Course: Data Analysis and Fintech Fundamentals (1001)

Group Members:

Ziheng LYU 2330036105

Shuoyan Zhang 2330036190

Songhao Deng 2330036015

Xin Wu 2330036159

Zeming CHEN 2330036010

Jiaming LIU 2330036089

Yitong LI 2330036070

Tutor: Dr. Peimin CHEN

Due date: May. 31

Contents

1 Introduction **4**

2 Data Ingestion **5**

2.1 Data Collection Strategy 5

2.1.1 Financial News Data 5

2.1.2 Market OHLCV Data 5

2.2 Automation and Scheduling Mechanism 5

2.3 Data Quality Control and Logging 5

3 Secure Database and Data Architecture Design **6**

3.1 Introduction and Purpose of the Section 6

3.2 Conceptual Foundation 6

3.3 Methodological and Design Framework 7

3.4 System Architecture Diagram 7

3.5 Layered Data Architecture 8

3.5.1 Bronze Layer: Raw Immutable Evidence 8

3.5.2 Silver Layer: Validated and Bias-Safe Analytical Data 9

3.5.3 Gold Layer: Model-Ready Features and System Logs 9

3.6 Logical Schema Design 10

3.7 Data Update, Deduplication, and Lineage 12

3.8 Temporal Validity and Look-Ahead Bias Control 12

3.9 Secure Communication and Integrity Protection 13

3.10 Reliability, Availability, and Scalability Logic 14

3.11 Data Access Layer and Module Integration 15

3.12 Interpretation, Implications, and Critical Discussion 16

3.13 Limitations and Assumptions 17

3.14 Concluding Transition 17

4 NLP Pipeline **17**

4.1 Introduction 17

4.2 Data Preparation and Processing 18

4.3 Sentiment Method 18

4.3.1 Lexicon-based Method 18

4.3.2 Logistic Regression Method 19

4.3.3 FinBERT Pipeline: Architecture, Implementation, and Critical Challenges 22

4.4 Model Comparison and Performance Evaluation 25

4.4.1 Metric Definitions and Evaluation Framework 25

4.4.2 Composite Scoring Logic . . . 25
4.4.3 Comparative Analysis and Conclusions . . . 26

5 RL Trading Engine 26
5.1 Module Objective . . . 26
5.2 Data Setting and Sample Split . . . 27
5.3 Reinforcement Learning Environment Design . . . 28
5.4 Portfolio Update and Reward Function . . . 29
5.5 DQN Model Construction . . . 30
5.6 Exploration Strategy and Training Process . . . 31
5.6.1 Epsilon-Greedy Policy . . . 31
5.7 Ablation Experiment . . . 32
5.8 Evaluation Metrics . . . 32
5.9 Convergence Analysis . . . 33
5.9.1 Training Set Analysis . . . 33
5.10 Validation Set Analysis . . . 34
5.10.1 Validation Checkpoint Analysis . . . 35
5.11 Test Set Analysis . . . 35
5.12 Chapter Summary . . . 36

6 Front-End Dashboard Design and Implementation 36
6.1 Module Objective and Evaluation Context . . . 36
6.2 Front-End Architecture . . . 37
6.3 User Flow and Interface Structure . . . 38
6.4 Market Data and Technical Indicator Visualization . . . 39
6.5 Sentiment Layer Presentation . . . 40
6.6 RL Decision Layer and Interactive State Inspection . . . 41
6.7 Portfolio, Benchmark, and Ablation Views . . . 42
6.8 Pipeline Health and Reliability Monitoring . . . 44
6.9 Visual Design Choices . . . 44
6.10 Connection with the Current A-Share Environment . . . 45
6.11 Summary . . . 46

A Mathematical Process and Security Logic of ChaCha20–Poly1305 48
A.1 Introduction and Purpose of the Appendix . . . 48
A.2 Conceptual Foundation: AEAD in the Project Communication Layer . . . 48
A.3 Overall Algorithmic Workflow . . . 49
A.4 ChaCha20 State Construction . . . 49
A.5 Quarter-Round Transformation . . . 50
A.6 Column Rounds, Diagonal Rounds, and Double Rounds . . . 51

A.7 ChaCha20 Encryption and Correctness Proof 51
A.8 Necessity of Nonce Uniqueness 52
A.9 Poly1305 One-Time Key Generation 52
A.10 Poly1305 Message Encoding 53
A.11 Poly1305 Arithmetic over the Field $2^{130} - 5$ 53
A.12 Polynomial Expansion and Integrity Proof Sketch 54
A.13 Authentication Verification and Decryption Procedure 55
A.14 Integrated AEAD Security Logic 55
A.15 Diagrammatic Representation of the Encryption Pipeline 56
A.16 Implications for the Project's Data Architecture 56
A.17 Limitations and Boundary Conditions 56
A.18 Concluding Transition 58

1 Introduction

In our modern era, the semiconductor industry is specialized by high volatility, rapid technological iteration, and sensitivity to macroeconomic and policy changes. Traditional quantitative trading strategies rely heavily on technical indicators and historical price patterns, which often fail to capture market sentiment and unstructured information embedded in financial news. Therefore, to tackle these technical problems, this project designs and implements a modular, end-to-end intelligent trading platform that integrates Data Digestion, Natural Language Processing (NLP), Data Storage, Reinforcement Learning (RL), and Fron-end Engine. The platform targets the South China CSI Semiconductor ETF (ticker: **159325.SZ**), aiming to improve trading decision quality by a core trading engine incorporating real-time market sentiment and dynamic learning mechanisms.

The proposed system follows a data-driven pipeline consisting of five interconnected modules: data ingestion, NLP pipeline, data storage, RL trading engine, and front-end visualization. Each module is designed for scalability and maintainability, making sure that seamless data flow from raw information collection to actionable trading signals.

Specifically, the primary objectives of this project are shown below, summarizing each module's work:

1. Automate the collection and preprocessing of multi-source financial and market data.
2. Extract sentiment signals from unstructured financial news using advanced NLP techniques.
3. Establish a robust data storage framework for structured and unstructured data management.
4. Develop a reinforcement learning-based trading engine that adapts to changing market conditions.
5. Build an interactive front-end dashboard for strategy monitoring, performance analysis, and result visualization.

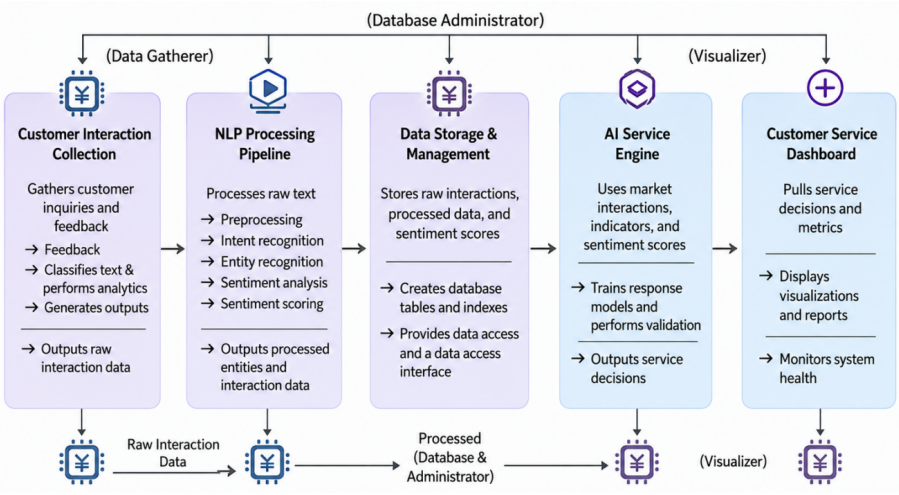


Figure 1: Workflow of the NLP Sentiment Analysis Pipeline

2 Data Ingestion

2.1 Data Collection Strategy

Data ingestion serves as the foundation of the entire system, providing high-quality and time-consistent raw data for the preparation for subsequent analysis and modeling. The dataset is divided into two main categories: financial news data and market OHLCV data.

2.1.1 Financial News Data

Financial news is a vital source of market sentiment and trend information. The collection focuses on authoritative and industry-specific content especially related to the whole supply chain and specific companies:

- **Data Sources**: Official semiconductor industry announcements, and financial news portals.
- **Content Scope**: Industry policy updates, corporate earnings reports, technological breakthroughs, supply chain dynamics, and macroeconomic news affecting the semiconductor sector.
- **Time Division**: Data from January 2025 to December 2025 is used for model training, while data from January 2026 to April 2026 is reserved for automate testing and simulation, ensuring temporal consistency and avoiding bias.

2.1.2 Market OHLCV Data

Market data provides the quantitative basis for trading strategy design and model training:

- **Data Source**: Akshare API, a reliable open-source financial data interface.
- **Time Range**: January 1, 2025, to April 30, 2026, covering both training and testing periods.
- **Key Features**: Daily open price, high price, low price, close price, trading volume, and back-adjusted close price. It should be noticed that we used adjusted price ensuring continuity in historical price series, eliminating distortions caused by splits, dividends, and other corporate actions.

2.2 Automation and Scheduling Mechanism

To get real-time data and minimize manual work, the data collection process we finalized is fully automatic. System scheduling is designed for run data collection scripts every trading day, gathering news content, collecting market data, conducting basic validation and saving data automatically with regular data updates to support long-term use or data longevity such as calculating long-term price average.

2.3 Data Quality Control and Logging

Data quality is important for reliable model training and trading decisions, so strict quality control process is integrated into the main workflow where the system checks for missing values, outliers, and duplicates. Here, missing values are fixed or filled with time-based interpolation, outliers are recognized by standard statistical checking, and duplicates are eliminated using

unique identifiers. In additions, to prevent data persistency, a detailed logging system records all key information, including collection time, data volume, API status, errors, and quality results. Therefore, pre-processed data is stored in folders, prepared for data processing, storage, natural language processing and reinforce-learning.

3 Secure Database and Data Architecture Design

3.1 Introduction and Purpose of the Section

This section presents the database and secure data-infrastructure design for the proposed NLP-driven reinforcement learning trading platform. The overall project requires an end-to-end pipeline in which raw financial text and market data are collected, transformed into usable signals, stored in a persistent database layer, and then consumed by the trading engine and monitoring dashboard. Within this pipeline, the data storage module is responsible for persisting market data, news records, sentiment signals, and trading logs, while also providing an access layer through SQLite or PostgreSQL-compatible database operations. The project also requires that the final system integrate all modules into a runnable platform and that the reinforcement learning state vector include price indicators, portfolio variables, and sentiment information.

The central purpose of the database design is therefore not limited to data preservation. It must also guarantee that data are structured, traceable, temporally valid, securely transmitted, and efficiently retrievable. These requirements are especially important in a trading-system context because incorrect timestamps, duplicated records, corrupted packets, or inconsistent feature construction may directly distort model training, backtesting, and performance evaluation. Consequently, the database layer is designed as a secure data architecture rather than a conventional storage table collection.

The proposed infrastructure consists of distributed edge collectors, encrypted communication channels, a central Kafka message layer, a hybrid PostgreSQL/TimescaleDB relational database, and a Parquet-based data lake. The system architecture includes edge collection nodes for Shenzhen and Shanghai market-side data, a central Kafka cluster with topics such as `market.bars`, `market.ticks`, `news.raw`, `sentiment.signals`, and `trading.logs`, and a central storage layer containing PostgreSQL, TimescaleDB, and a Parquet lake.

3.2 Conceptual Foundation

The database architecture is developed from three core principles: layered data governance, time-series consistency, and secure transmission.

The first principle is **layered data governance**. Raw source records, cleaned analytical records, and model-ready features should not be stored as if they have the same epistemic status. A raw news response or order-book message is an observation; a cleaned news table is a validated representation of that observation; an RL state vector is a derived modeling object. Mixing these levels in a single table would make audit, reconstruction, and error diagnosis difficult. Therefore, the system adopts a Bronze–Silver–Gold structure: Bronze stores immutable raw records, Silver stores normalized and bias-safe records, and Gold stores model-ready features

and trading logs. The database design explicitly follows this hybrid lake-house and relational time-series logic.

The second principle is **time-series consistency**. Financial data are meaningful only when their temporal availability is defined precisely. For example, a news article may have a publication timestamp, an availability timestamp, and an ingestion timestamp. These times are not interchangeable. A trading decision at time τ may only use information that was available before τ. This requirement is directly connected to the project's walk-forward validation and no-look-ahead-bias requirement.

The third principle is **secure transmission**. Edge nodes communicate with the central platform through VPN and TLS-protected channels. The design uses a WireGuard-like logic: asymmetric key exchange establishes symmetric session keys, and encrypted packet transmission uses authenticated encryption. In the current design, ChaCha20-Poly1305 is used for VPN packet confidentiality and integrity, while TLS 1.3 is used for TCP/IP and SFTP communication.

3.3 Methodological and Design Framework

The database architecture can be understood as a transformation chain from source events to validated model inputs. Let $x_{i,t}$ denote a raw event collected by edge node i at time t. The event may be a market bar, tick message, order-book snapshot, raw news record, sentiment signal, or trading log. The data pipeline can be represented as:

$$x_{i,t} \xrightarrow{\phi_{\text{edge}}} m_{k,t} \xrightarrow{\phi_{\text{kafka}}} b_t \xrightarrow{\phi_{\text{silver}}} s_t \xrightarrow{\phi_{\text{gold}}} g_t.$$

Here, ϕ_{edge} denotes edge-side collection and message construction, $m_{k,t}$ denotes a Kafka message assigned to topic k, b_t denotes a Bronze-layer raw record, s_t denotes a Silver-layer validated record, and g_t denotes a Gold-layer feature or log record. This notation clarifies that the database process is not a simple insertion operation. It is a sequence of controlled transformations, each with a distinct role in reliability, interpretability, and downstream model validity.

The total data-processing latency can be decomposed as:

$$L_{\text{total}} = L_{\text{fetch}} + L_{\text{spool}} + L_{\text{vpn}} + L_{\text{kafka}} + L_{\text{write}} + L_{\text{feature}}.$$

In this expression, L_{fetch} is the time required to obtain data from the external source, L_{spool} is any waiting time in the local retry spool, L_{vpn} is secure transmission latency, L_{kafka} is message-broker latency, L_{write} is database or lake write latency, and L_{feature} is the time required to generate Silver or Gold outputs. This decomposition is useful because it identifies where delay may occur and why the architecture separates edge collection, Kafka buffering, and database persistence.

3.4 System Architecture Diagram

Figure 2 illustrates the relationship between edge collection, secure network transmission, central message buffering, and layered storage. Edge nodes reduce source-side collection delay,

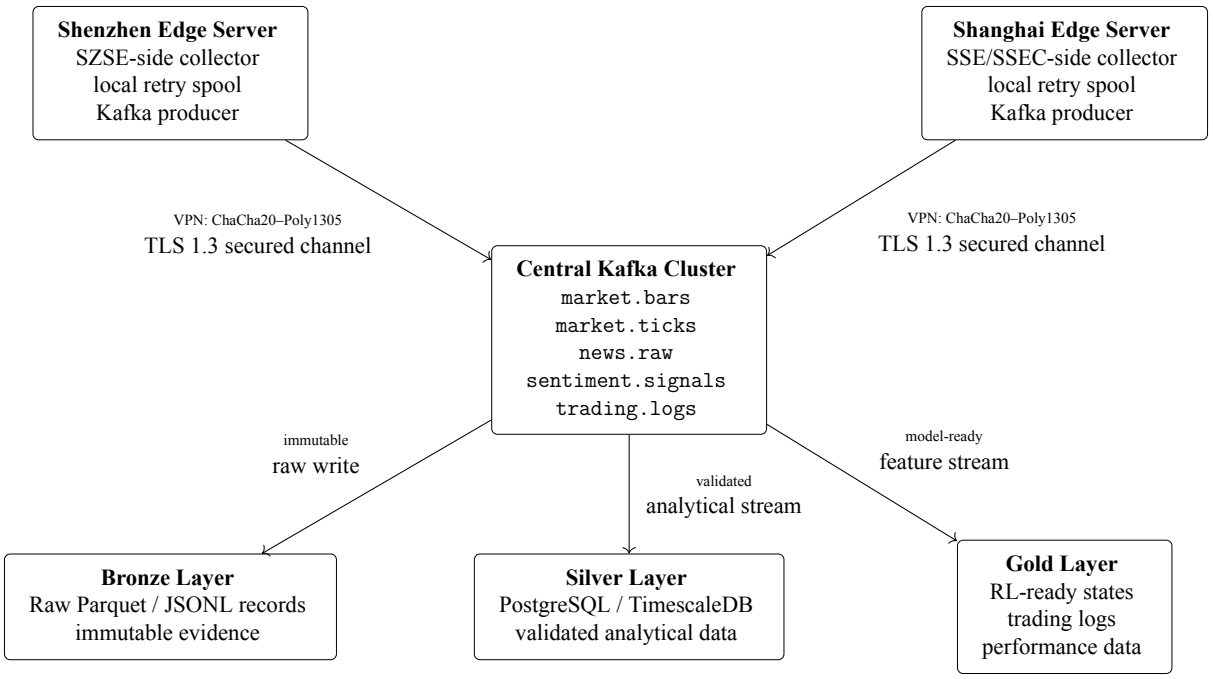


Figure 2: Secure distributed data architecture connecting edge collectors, encrypted transmission channels, central Kafka topics, and the Bronze–Silver–Gold storage layers.

Kafka decouples ingestion from downstream processing, and the three-layer storage architecture separates raw evidence, validated records, and model-facing outputs.

3.5 Layered Data Architecture

3.5.1 Bronze Layer: Raw Immutable Evidence

The Bronze layer stores records exactly as they are received from external APIs, exchange-side collectors, or upstream modules. It is append-only and should not overwrite source data. The design includes Bronze partitions for market bars, market ticks, and raw news, with directory structures such as `data_lake/bronze/market_bar/exchange=SZSE/interval=1d/date=...` and `data_lake/bronze/news_raw/source=sina_finance/date=...`.

Formally, for a raw record r_t, the Bronze update rule is:

$$B_{t+1} = B_t \cup \{r_t\}.$$

This equation expresses immutability. If a source later sends a corrected record, the correction should be appended as a new record rather than destructively replacing the previous one. The reason is methodological rather than merely technical: model experiments must be reproducible. If later transformations change, Silver and Gold layers can be rebuilt from Bronze evidence.

A Bronze record should preserve the original payload, source identifier, site identifier, event timestamp, ingestion timestamp, and payload hash. For example, the raw order-book example contains exchange, ticker, timestamp, source, site identifier, sequence number, availability timestamp, bid levels, and ask levels. This structure is suitable for raw storage because

it captures the source event without prematurely flattening or discarding information.

3.5.2 Silver Layer: Validated and Bias-Safe Analytical Data

The Silver layer converts raw records into standardized, validated, and queryable tables. It contains cleaned market bars, ticks, order-book snapshots, news records, ticker mentions, and sentiment signals. The Silver layer is described as high-quality and bias-safe storage for market data, news, sentiment signals, and trading logs.

The Silver transformation can be represented as:

$$S(r) = V(N(D(r))),$$

where $D(\cdot)$ removes or flags duplicates, $N(\cdot)$ normalizes field names and timestamp formats, and $V(\cdot)$ validates the record against schema and quality rules. This formula is useful because it separates three different operations that are often conflated. Deduplication decides whether two records are the same event; normalization makes records comparable across sources; validation determines whether a record is admissible for analytical use.

A Silver order-book snapshot, for example, should not only preserve the raw bid and ask structure but also compute deterministic microstructure variables such as best bid, best ask, spread, mid-price, total bid volume, total ask volume, and depth. The example Silver order-book schema includes these fields, together with `event_time`, `available_at`, `ingested_at`, `bronze_record_id`, `payload_hash`, and `quality_status`.

3.5.3 Gold Layer: Model-Ready Features and System Logs

The Gold layer stores derived features, RL state vectors, trading decisions, portfolio snapshots, performance metrics, and ablation results. Its purpose is not raw preservation but controlled downstream consumption. The design includes Gold directories for market features, sentiment features, RL states, agent decisions, portfolio snapshots, performance metrics, and ablation results.

The Gold transformation is:

$$G_\tau = F(S_{\leq \tau}),$$

where F is a feature construction function and $S_{\leq \tau}$ denotes Silver records available no later than decision time τ. This condition is crucial because features must not be constructed using information unavailable at the time of the modeled decision.

For RL integration, the Gold state vector must include the required project fields:

$$z_\tau = [\text{price}, \text{MA50}, \text{MA200}, \text{RSI}, \text{MACD}, \text{position}, \text{cash}, \text{sentiment_score}]\,.$$

The database layer supports this requirement by storing both the ordered `state_vector` and the corresponding `state_vector_name`, ensuring that downstream models consume features in a consistent order.

3.6 Logical Schema Design

The relational schema is organized around natural entities and their update behavior. Market bars and ticks are time-series records; news records are document-level objects; sentiment signals are model outputs; trading decisions and portfolio snapshots are experiment-level logs. The storage prompt also requires that the database module include database choice, table structure, data access layer, news table, market-data table, sentiment-signal table, trading-log table, deduplication keys, and index design.

Table 1: Logical schema and recommended keys.

Table	Layer	Purpose	Recommended key
dim_instrument	Silver	Security metadata	UNIQUE(ticker, exchange)
silver_market_bar	Silver	Clean OHLCV records	UNIQUE(ticker, exchange, bar_interval, ts, source)
silver_market_tick	Silver	Tick-level trades/quotes	UNIQUE(ticker, exchange, ts, seq_no, source)
silver_order_book_snapshot	Silver	Snapshot summary	PRIMARY KEY(snapshot_id)
silver_order_book_level	Silver	Bid/ask level records	PRIMARY KEY(snapshot_id, side, level)
silver_news	Silver	Cleaned news documents	UNIQUE(url_hash)
silver_news_ticker_mention	Silver	News-to-ticker mapping	PRIMARY KEY(news_id, ticker, exchange)
silver_sentiment_signal	Silver	Sentiment model outputs	UNIQUE(ticker, exchange, signal_date, feature_window, sentiment_method, model_version)
gold_market_features_daily	Gold	Technical indicators	UNIQUE(ticker, exchange, date, feature_version)
gold_rl_state_daily	Gold	Model state vectors	UNIQUE(ticker, exchange, date, with_nlp_signal, feature_version)
gold_agent_decision	Gold	Agent actions and Q-values	UNIQUE(run_id, step_id, ticker, decision_time)
gold_portfolio_snapshot	Gold	Portfolio state	UNIQUE(run_id, ts)
gold_performance_metric	Gold	Evaluation metrics	UNIQUE(run_id, window_id, strategy_name)

The order-book schema requires special normalization. A raw order book contains repeated bid and ask levels. If this structure were stored only as JSON, level-wise liquidity analysis would become inefficient. Therefore, the snapshot-level and level-level components should be separated:

$$\text{Snapshot}_t = (\text{snapshot_id}, \text{ticker}, \text{ts}, \text{best_bid}, \text{best_ask}, \text{spread}, \text{mid}),$$

$$\text{Level}_{t,\ell} = (\text{snapshot_id}, \text{side}, \ell, \text{price}_{t,\ell}, \text{volume}_{t,\ell}).$$

This design supports microstructure calculations such as order-book imbalance:

$$\text{Imbalance}_t = \frac{\sum_{\ell=1}^{L} V_{t,\ell}^{\text{bid}} - \sum_{\ell=1}^{L} V_{t,\ell}^{\text{ask}}}{\sum_{\ell=1}^{L} V_{t,\ell}^{\text{bid}} + \sum_{\ell=1}^{L} V_{t,\ell}^{\text{ask}}}.$$

Here, L is the order-book depth, $V_{t,\ell}^{\text{bid}}$ is bid volume at level ℓ, and $V_{t,\ell}^{\text{ask}}$ is ask volume at level ℓ. The formula demonstrates why preserving level-specific bid and ask volumes is necessary: without the normalized level table, this feature cannot be computed reliably.

3.7 Data Update, Deduplication, and Lineage

The update mechanism must prevent duplicate storage while preserving corrections and auditability. Let $K(r)$ denote the natural key of record r, and let $h(r)$ denote the payload hash computed from a deterministic canonical serialization of the record:

$$h(r) = H(\text{canon}(r)).$$

The Silver-layer upsert rule is:

$$\text{upsert}(r) = \begin{cases} \text{insert}(r), & K(r) \notin T, \\ \text{ignore}(r), & K(r) \in T \wedge h(r) = h(T[K(r)]), \\ \text{insert_correction}(r), & K(r) \in T \wedge h(r) \neq h(T[K(r)]). \end{cases}$$

This rule is preferable to unconditional overwriting. If a record is identical to an existing record, it is ignored. If the natural key is the same but the payload differs, the difference is preserved as a correction or versioned record. This mechanism supports both operational deduplication and methodological auditability.

Lineage must be preserved from Bronze to Gold. For a Gold feature g, the lineage can be represented as:

$$\text{lineage}(g) = \{\text{source_tables}, \text{source_record_ids}, \text{feature_version}, \text{generated_at}\}.$$

This lineage is necessary because a Gold feature is not a directly observed fact. It is the result of one or more deterministic transformations applied to earlier records. Without lineage, model results cannot be traced back to the underlying data.

3.8 Temporal Validity and Look-Ahead Bias Control

A major design risk in financial machine learning is look-ahead bias. A database may be structurally correct yet analytically invalid if it allows downstream modules to access informa-

tion that was not available at the decision time. Therefore, the database must distinguish at least four timestamps:

$$t_{\text{event}}, \quad t_{\text{published}}, \quad t_{\text{available}}, \quad t_{\text{ingested}}.$$

For any decision time τ, a feature vector z_τ is admissible only if every source record used to construct it was available no later than τ:

$$z_\tau \text{ is admissible} \iff \max_{r \in \mathcal{S}(z_\tau)} t_{\text{available}}(r) \leq \tau.$$

Here, $\mathcal{S}(z_\tau)$ denotes the set of source records used to construct z_τ. This condition must be enforced in feature-generation SQL queries and stored as a validation field in the Gold layer. It also directly supports the required walk-forward validation procedure.

A walk-forward split can be represented as:

$$\mathcal{D}_{\text{train}}^{(j)} = [t_0, t_j], \qquad \mathcal{D}_{\text{test}}^{(j)} = (t_j, t_j + \Delta].$$

The training window ends before the test window begins, and feature availability constraints must be enforced in both windows. This design prevents evaluation results from being inflated by future information.

3.9 Secure Communication and Integrity Protection

The secure communication mechanism protects data before they enter the database. The infrastructure uses VPN encryption based on ChaCha20-Poly1305 and TLS 1.3 for TCP/IP and SFTP transmission. The design follows a two-stage logic: first, an asymmetric handshake establishes a shared secret; second, symmetric authenticated encryption protects packet transmission.

Let a and b be private scalars held by two communication endpoints, and let G be the elliptic-curve base point. Their public keys are:

$$A = aG, \qquad B = bG.$$

The two endpoints compute:

$$S_A = aB = a(bG) = abG,$$

$$S_B = bA = b(aG) = abG.$$

Thus:

$$S_A = S_B.$$

A symmetric session key is then derived:

$$K = \text{KDF}(S_A) = \text{KDF}(S_B).$$

After this step, ChaCha20-Poly1305 protects packets through authenticated encryption:

$$(C, T) = \text{AEAD.}\,\text{Enc}_{K,N}(P, A),$$

where P is plaintext, A is associated data, C is ciphertext, T is the authentication tag, K

is the session key, and N is a nonce. A packet is admitted only if:

$$\mathrm{Verify}_{K,N}(A, C, T) = 1.$$

This condition connects network security directly to database reliability. If verification fails, the packet must not be inserted into Kafka or persisted into Bronze storage. If verification succeeds, the packet can be processed further and assigned a database payload hash.

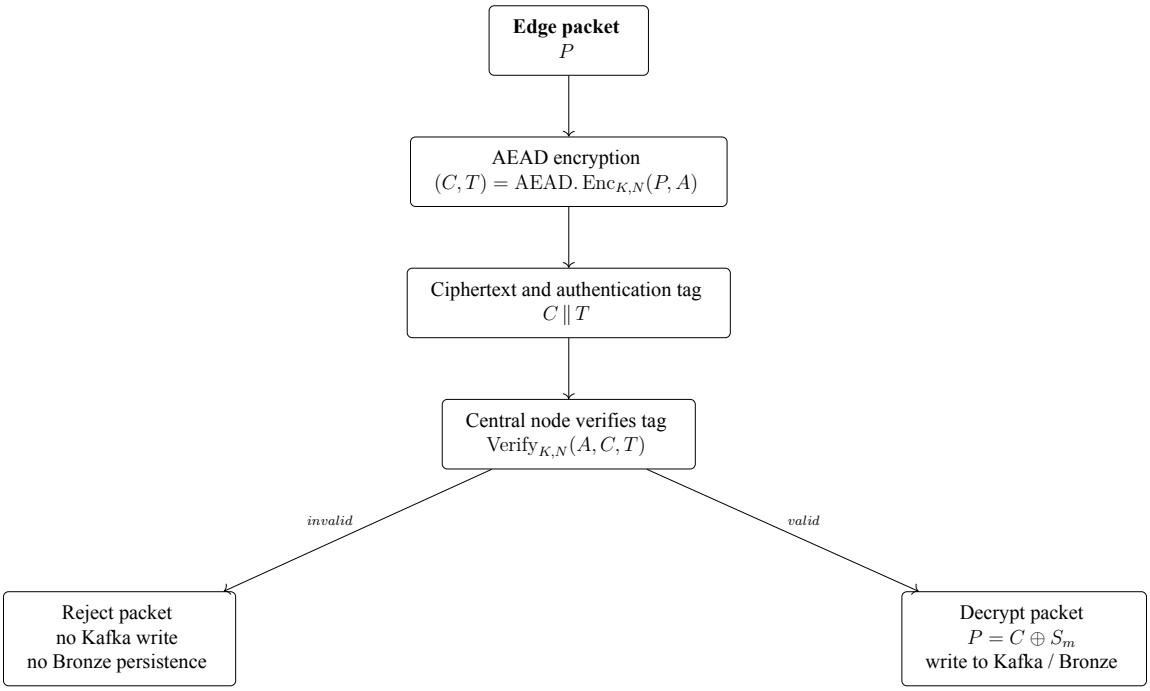


Figure 3: Secure packet admission workflow. A packet is admitted into the data pipeline only after ChaCha20–Poly1305 authentication succeeds.

Figure 3 ensures that the database layer receives only packets that satisfy cryptographic integrity verification. Database-level hashes and lineage then continue the integrity chain after ingestion.

3.10 Reliability, Availability, and Scalability Logic

Reliability is supported by the combination of local retry spools, Kafka buffering, and immutable raw storage. Let $\lambda_i(t)$ be the event arrival rate at edge node i, and let $\mu_i(t)$ be the successful transmission rate from node i to the central Kafka cluster. The retry-spool queue evolves as:

$$Q_i(t + \Delta) = \max\left(0, Q_i(t) + \int_t^{t+\Delta} (\lambda_i(u) - \mu_i(u))\, du\right).$$

The long-run stability condition is:

$$\mathbb{E}[\lambda_i] < \mathbb{E}[\mu_i].$$

This condition explains why the retry spool is a fault-tolerance mechanism but not a substitute for adequate network and Kafka capacity. If the average arrival rate permanently exceeds the transmission rate, the spool will eventually overflow.

Kafka scalability can be expressed through topic partitioning. For topic k, let p_k be the

number of partitions and $\mu_{k,j}$ be the processing capacity of partition j. The total service capacity is:

$$\mu_k = \sum_{j=1}^{p_k} \mu_{k,j}.$$

The topic remains stable when:

$$\lambda_k < \mu_k.$$

This condition supports the use of multiple Kafka topics and partitions. Market bars, ticks, news, sentiment signals, and trading logs have different arrival rates and consumption patterns; separating them into topics prevents one data type from blocking all others.

3.11 Data Access Layer and Module Integration

A controlled data access layer is necessary because different modules must read and write database records consistently. Direct SQL scattered across modules would increase the risk of inconsistent timestamp filters, duplicate logic, or feature-order mismatch. The storage module should therefore expose repository-style interfaces rather than unstructured database access.

A suitable implementation structure is:

```
src/
  storage/
    connection.py
    schema.py
    repositories.py
    feature_store.py
    validation.py
    migration.py
```

The main interfaces are:

```
MarketRepository:
  upsert_market_bars()
  upsert_market_ticks()
  load_market_features()

NewsRepository:
  upsert_news()
  upsert_ticker_mentions()
  load_news_for_nlp()

SentimentRepository:
  upsert_sentiment_signals()
  load_sentiment_features()
```

```
FeatureStore:
  build_market_features_daily()
  build_sentiment_features_daily()
  build_rl_state_daily()
  load_state_vectors()

TradingLogRepository:
  write_agent_decision()
  write_portfolio_snapshot()
  write_performance_metric()
```

This structure preserves module boundaries. The database module stores and serves data; it does not perform sentiment inference, train the RL agent, or implement dashboard rendering. This boundary is consistent with the project's modular design requirement, where Data Ingestion, NLP Pipeline, Data Storage, RL Trading Engine, and Front-End Dashboard are separate responsibilities.

3.12 Interpretation, Implications, and Critical Discussion

The proposed architecture improves the validity and reliability of the trading platform in three ways. First, the Bronze–Silver–Gold separation improves reproducibility. Raw records are preserved, cleaned records are standardized, and model-facing features are versioned. This separation makes it possible to reconstruct feature tables when cleaning rules, sentiment methods, or technical-indicator definitions change.

Second, the architecture improves temporal validity. By requiring `available_at` and enforcing the condition $\max t_{\text{available}} \leq \tau$, the database design directly supports no-look-ahead backtesting. This is not merely a database optimization; it is a condition for credible financial evaluation.

Third, the security architecture improves data trustworthiness before storage. ChaCha20-Poly1305 authentication prevents modified packets from entering the data pipeline, while database-level hashes preserve integrity after ingestion. These mechanisms operate at different layers but jointly support the same objective: preventing corrupted or unauthorized data from influencing downstream model decisions.

The design also has a practical scalability implication. PostgreSQL and TimescaleDB support structured and time-series queries, while Parquet supports historical reconstruction and large-scale storage. This hybrid design is more complex than a single SQLite file, but the additional complexity is justified by the project's need to handle market bars, ticks, order-book snapshots, news documents, sentiment signals, trading logs, portfolio snapshots, and experiment-level metrics.

3.13 Limitations and Assumptions

Several limitations should be acknowledged. First, the architecture assumes that edge collectors can maintain stable identifiers, accurate timestamps, and monotonic sequence numbers. If source timestamps are unreliable, the database can preserve and validate them, but it cannot fully infer the correct event chronology without additional external information.

Second, the security argument assumes correct key management and nonce discipline. ChaCha20-Poly1305 provides confidentiality and integrity only if keys remain secret and nonces are not reused under the same key. Implementation errors in session management would weaken the intended protection.

Third, the proposed hybrid architecture is more complex than a minimal course-project database. It requires schema migration, feature versioning, object-storage organization, and repository-level access control. This complexity is appropriate for a realistic fintech platform, but it increases implementation workload.

Fourth, the database layer supports downstream NLP and RL modules but does not guarantee their modeling performance. It ensures that features are structured, available, and traceable; the predictive or trading value of those features must still be evaluated through the required experimental design.

3.14 Concluding Transition

This section has reconstructed the database and secure data architecture as a unified infrastructure layer for the trading platform. The design begins with distributed edge collection, protects transmission through VPN and TLS mechanisms, buffers messages through Kafka, stores immutable raw evidence in the Bronze layer, normalizes validated analytical records in the Silver layer, and generates model-ready features and trading logs in the Gold layer. The use of formal data-flow, latency, queueing, integrity, and temporal-validity representations clarifies why each component is necessary and how it contributes to the overall project objective.

The resulting architecture provides the foundation for downstream modeling and evaluation. The next section can therefore proceed from infrastructure design to the implementation of feature construction, trading-state generation, or experimental evaluation, with the assurance that the underlying data pipeline has been structured to support reliability, traceability, security, and no-look-ahead validation.

4 NLP Pipeline

4.1 Introduction

The objective of the NLP module is to transform Chinese A-share semiconductor industry news into a structured daily sentiment score for Semiconductor ETF Southern 159325. This score can then be merged with ETF price data and used as part of the reinforcement learning trading state vector.

4.2 Data Preparation and Processing

The input dataset contains 320 original synthetic Chinese financial news articles related to the A-share semiconductor industry. Each row includes date, ETF code, ETF name, title, full article text, and a manually assigned sentiment label: positive, neutral, or negative. The NLP output is an ETF-level daily sentiment score.

The preprocessing pipeline includes text cleaning, Chinese tokenization, stopword removal, and lemmatization. The code uses the `jieba` and `Paddle LAC` packages to conduct these operations.

4.3 Sentiment Method

Here, we use three different sentiment analysis methods: Lexicon-based, Logistic Regression, and FinBERT. After running each method's program, we calculate the composite score using accuracy, macro precision, macro recall, macro F1, and weighted F1. Then, we select the method with the best overall performance, and the final sentiment score is obtained using the score of the best method.

4.3.1 Lexicon-based Method

The lexicon-based method classifies sentiment by matching words in each article against a predefined sentiment dictionary. This approach is widely used in sentiment analysis because it is simple, transparent, and easy to interpret (Taboada et al., 2011). In financial text, however, general-purpose sentiment dictionaries may not perform well because words can have different meanings in business and financial contexts. For example, Loughran and McDonald (2011) show that finance-specific dictionaries are more appropriate for financial textual analysis than general sentiment word lists.

Therefore, this project uses a Chinese financial sentiment dictionary tailored to A-share semiconductor news. Examples of positive and negative financial terms are shown in Table 2.

Table 2: Examples of Chinese Financial Sentiment Terms

Sentiment Type	**Example Terms**
Positive	上涨, 利好, 订单增长, 需求回暖, 国产替代, 突破, 放量, 盈利改善, 毛利率提升
Negative	下跌, 利空, 需求疲软, 库存压力, 价格下行, 竞争加剧, 亏损, 利润下滑, 估值压力

Lexicon-based Sentiment Score Formula

For article i, let:

$$P_i = \text{number of positive sentiment words in article } i$$

$$N_i = \text{number of negative sentiment words in article } i$$

The lexicon-based sentiment score is calculated as:

$$S_i^{\text{lexicon}} = \frac{P_i - N_i}{P_i + N_i}$$

where:

$$S_i^{\text{lexicon}} \in [-1, 1]$$

If no positive or negative words are detected, then:

$$P_i + N_i = 0$$

and the score is set as:

$$S_i^{\text{lexicon}} = 0$$

This means the article is treated as neutral.

The sentiment label is assigned using the following rule:

- If the sentiment score is greater than 0.05, the article is classified as Positive.
- If the sentiment score is between -0.05 and 0.05, the article is classified as Neutral.
- If the sentiment score is less than -0.05, the article is classified as Negative.

For example, if one article contains 8 positive words and 3 negative words:

$$S_i^{\text{lexicon}} = \frac{8 - 3}{8 + 3} = \frac{5}{11} = 0.455$$

Since $0.455 > 0.05$, the article is classified as positive.

In this project, the lexicon-based method serves as a baseline sentiment model. Its main advantage is interpretability. If an article is classified as positive, the result can be explained by identifying positive financial words such as "订单增长", "需求回暖", or "盈利改善". This makes the method useful for explaining the basic logic of sentiment scoring in the report.

However, the method also has limitations. A fixed word dictionary may fail to understand negation, context, and complex financial expressions. For example, the phrase "库存压力缓解" contains "库存压力", which is negative by itself, but the overall phrase is positive because the pressure is reducing. This limitation is consistent with prior sentiment analysis literature, which notes that keyword-based approaches are less flexible than statistical or machine learning methods (Pang & Lee, 2008; Taboada et al., 2011).

4.3.2 Logistic Regression Method

Logistic Regression is implemented as a supervised machine learning approach for sentiment classification. Unlike the lexicon-based method, Logistic Regression does not rely only on a manually created sentiment dictionary. Instead, it learns the relationship between article features and sentiment labels from labelled training data.

The input data contains Chinese A-share semiconductor articles with manually assigned sentiment labels:

`positive`, `neutral`, `negative`

In text classification, TF-IDF is commonly used to convert text into numerical feature

vectors by giving higher weights to terms that are important in a document but not too common across the entire corpus (Manning et al., 2008). Logistic Regression and maximum entropy models have also been widely used for text classification tasks (Nigam et al., 1999).

TF-IDF Feature Representation

Because machine learning models cannot directly process raw text, each article is first transformed into a numerical vector using TF-IDF.

The term frequency of word t in document d is:

$$\mathrm{TF}(t,d) = \frac{f_{t,d}}{\sum_k f_{k,d}}$$

where $f_{t,d}$ is the number of times word t appears in document d, and $\sum_k f_{k,d}$ is the total number of words in document d.

The inverse document frequency is:

$$\mathrm{IDF}(t) = \log\left(\frac{N}{\mathrm{DF}(t)}\right)$$

where N is the total number of documents, and $\mathrm{DF}(t)$ is the number of documents containing word t.

The final TF-IDF weight is:

$$\text{TF-IDF}(t,d) = \mathrm{TF}(t,d) \times \mathrm{IDF}(t)$$

In this project, words such as “利好”, “订单增长”, “库存压力”, and “价格下行” are expected to have stronger sentiment-related information than common words such as “公司”, “市场”, or “行业”.

Logistic Regression Model Formula

For binary classification, Logistic Regression estimates the probability of a positive class using the sigmoid function:

$$P(y=1 \mid x) = \frac{1}{1+e^{-z}}$$

where:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k$$

Here, $x_1, x_2, \ldots, x_k$ are TF-IDF features, and $\beta_0, \beta_1, \ldots, \beta_k$ are model parameters learned from training data.

Since this project uses three sentiment classes, namely positive, neutral, and negative, a multinomial Logistic Regression model is used. The predicted probability for class c is calculated using the softmax function:

$$P(y=c \mid x) = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}}$$

where:

$$z_c = \beta_{c0} + \beta_{c1} x_1 + \beta_{c2} x_2 + \cdots + \beta_{ck} x_k$$

and $c = 3$.

The predicted label is the class with the highest predicted probability:

$$\hat{y}_i = \arg\max_c P(y = c \mid x_i)$$

The Logistic Regression model in this project can be implemented using scikit-learn, which provides standard machine learning tools for supervised learning, model training, and evaluation (Pedregosa et al., 2011).

Logistic Regression Sentiment Score

To make the Logistic Regression output usable for the reinforcement learning module, the predicted probabilities are converted into a continuous sentiment score.

Let:

$$P_i^{\text{pos}} = P(y_i = \text{positive} \mid x_i)$$
$$P_i^{\text{neu}} = P(y_i = \text{neutral} \mid x_i)$$
$$P_i^{\text{neg}} = P(y_i = \text{negative} \mid x_i)$$

The Logistic Regression sentiment score is defined as:

$$S_i^{\text{LR}} = P_i^{\text{pos}} - P_i^{\text{neg}}$$

This score is approximately bounded by:

$$S_i^{\text{LR}} \in [-1, 1]$$

A value close to 1 indicates strongly positive sentiment, a value close to -1 indicates strongly negative sentiment, and a value close to 0 indicates neutral or uncertain sentiment.

For example, if the model outputs:

$$P_i^{\text{pos}} = 0.70, \quad P_i^{\text{neu}} = 0.20, \quad P_i^{\text{neg}} = 0.10$$

then:

$$S_i^{\text{LR}} = 0.70 - 0.10 = 0.60$$

The article is therefore classified as positive and receives a positive sentiment score.

Model Training Objective

The Logistic Regression model is trained by minimizing the cross-entropy loss. For multi-class classification, the loss function is:

$$\mathcal{L} = -\sum_{i=1}^{n} \sum_{c=1}^{C} y_{i,c} \log\left(P(y_i = c \mid x_i)\right)$$

where n is the number of training articles, C is the number of sentiment classes, and $y_{i,c}$ equals 1 if article i belongs to class c, and 0 otherwise.

The objective is to learn model parameters that assign high predicted probabilities to the correct sentiment labels.

4.3.3 FinBERT Pipeline: Architecture, Implementation, and Critical Challenges

The FinBERT framework transforms unstructured financial text into actionable sentiment signals through a rigorously structured three-phase lifecycle: self-supervised pre-training, supervised fine-tuning, and real-time inference. Each phase is governed by specific mathematical objectives, hyperparameter constraints, and engineering safeguards to ensure robustness in production environments.

Phase 1: Masked Language Model (MLM) Pre-training

Phase 1 operates under a self-supervised paradigm, enabling the model to learn domain-specific financial semantics from massive unlabeled corpora (e.g., SEC filings, Reuters, Bloomberg) without human annotation.

Process and Mathematical Formulation

The pipeline begins by randomly masking 15% of tokens in each sequence. Following the original BERT protocol, 80% of masked tokens are replaced with `[MASK]`, 10% are replaced with random words, and 10% remain unchanged. This strategy prevents distribution shift between pre-training and downstream fine-tuning. The model processes the masked sequence through a 12-layer bidirectional Transformer encoder. Multi-head self-attention computes pairwise token relevance:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad \text{where } Q = XW^Q, K = XW^K, V = XW^V. \tag{1}$$

The probability of the true masked word w_i given the bidirectional context $w_{\setminus M}$ is projected back to the vocabulary space:

$$P(w_i \mid w_{\setminus M}; \theta) = \frac{\exp(h_i \cdot e_{w_i})}{\sum_{w \in V} \exp(h_i \cdot e_w)}. \tag{2}$$

The training objective minimizes the MLM cross-entropy loss to update parameters θ:

$$\mathcal{L}_{\text{MLM}} = -\sum_{i \in M} \log P(w_i \mid w_{\setminus M}; \theta). \tag{3}$$

Key Hyperparameters and Implementation Notes

- **Learning Rate:** 1×10^{-4} to 5×10^{-5} with linear warmup (10k steps). A relatively larger LR enables broad exploration of the semantic manifold during initial training.
- **Batch Size:** 256–512 (achieved via gradient accumulation). Large batches stabilize representation learning and maximize GPU throughput.
- **Sequence Length:** Truncated/padded to 512 tokens. Longer sequences incur quadratic attention complexity $O(L^2)$, making them computationally prohibitive.
- **Precision:** FP16/BF16 mixed precision with gradient scaling. Cuts VRAM usage by $\sim 50\%$ and accelerates training 2–3$\times$ without accuracy degradation.

- **Iteration Limit:** $\sim$1,000,000 steps or 1–3 epochs. Training is monitored via validation perplexity; early stopping triggers if perplexity plateaus for > 50k steps.

Critical Challenges

The primary engineering focus is scale and stability. The model must learn deep contextual dependencies without memorizing surface forms. Mixed precision and large batch training are mandatory to handle the computational load, while the 15% masking distribution strictly preserves alignment with downstream tokenization formats.

Phase 2: Supervised Fine-tuning for Sentiment Classification

Phase 2 transitions from language modeling to task-specific sentiment classification. The pre-trained weights θ^* are loaded, and a task-specific classification head (`Linear(768→3) + Softmax`) is appended. The model is trained on a curated dataset of labeled financial texts (d_i, y_i) where $y_i \in \{+1, 0, -1\}$.

Process and Mathematical Formulation

Input sequences are tokenized identically to Phase 1, with a special `[CLS]` token prepended. After 12 transformer layers, the hidden state at the `[CLS]` position, $h_{[CLS]} \in \mathbb{R}^{768}$, aggregates document-level semantics. The classification head projects this vector to logits and normalizes them:

$$\hat{y} = \text{softmax}(W_c h_{[CLS]} + b_c), \quad W_c \in \mathbb{R}^{3\times 768}. \tag{4}$$

The model minimizes the cross-entropy loss to learn the sentiment decision boundary:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{3} y_{i,k}\log \hat{y}_{i,k}. \tag{5}$$

Key Hyperparameters and Implementation Notes

- **Learning Rate:** 2×10^{-5} (AdamW with linear warmup). Critically, this rate is $10\times$ smaller than in Phase 1 to prevent catastrophic forgetting of financial semantic priors.
- **Epochs:** 3–5. BERT fine-tuning converges rapidly; exceeding 5 epochs typically leads to overfitting on small financial datasets.
- **Weight Decay:** $\lambda = 0.01$. Provides L2 regularization to control model complexity and prevent overfitting to noisy NLP features.
- **Batch Size:** 16–32. Constrained by GPU memory; smaller batches introduce implicit noise that can act as a regularizer.
- **Validation Strategy:** Walk-forward cross-validation. Financial data is non-stationary; strict temporal splitting $[t_0, t] \rightarrow [t, t + \Delta]$ is mandatory to prevent look-ahead bias.
- **Gradient Clipping:** Max norm $= 1.0$. Stabilizes backpropagation, particularly when processing long financial disclosures.

Critical Challenges

The core challenge is **catastrophic forgetting**: aggressive optimization can overwrite the nuanced financial semantics learned during pre-training. Mitigation requires strict learning rate control, early stopping based on validation Macro-F1 (not training loss), and handling class imbalance (e.g., rare bearish news) via weighted loss or focal loss. Additionally, temporal validation is non-negotiable to ensure out-of-sample generalization in live markets.

Phase 3: Inference and Real-Time Signal Generation

In production, the trained model is deployed for deterministic, low-latency inference. All training overheads are stripped, and parameters are frozen. The pipeline converts raw financial streams into discrete trading signals within strict SLA constraints.

Process and Mathematical Formulation

Incoming text undergoes identical tokenization and forward propagation through the frozen Transformer. The classification head outputs a probability distribution $\hat{y} = [P_+, P_0, P_-]$. To reduce single-article noise, scores are aggregated over a time window t:

$$s_t = \frac{1}{n} \sum_{i=1}^{n} \text{FinBERT}(h_t^{(i)}). \tag{6}$$

The aggregated score s_t is mapped to a trading signal using out-of-sample calibrated thresholds $\tau^{\pm}$:

$$\text{Signal}_t = \begin{cases} +1 & s_t > \tau^+ \quad \text{(Long)} \\ -1 & s_t < \tau^- \quad \text{(Short)} \\ 0 & \text{otherwise} \quad \text{(Hold)} \end{cases} \tag{7}$$

Key Hyperparameters and Implementation Notes

- **Execution Mode:** `model.eval()` with `torch.no_grad()`. Disables backpropagation and gradient tracking, reducing memory and compute by 60–70%.
- **Model Optimization:** DistilBERT architecture + INT8 quantization + ONNX/TensorRT runtime. Cuts latency by 60–80% and VRAM by $\sim 75\%$, essential for meeting the < 50 second end-to-end constraint.
- **Batching Strategy:** Dynamic batching with a 50ms timeout window. Groups asynchronous news arrivals to maximize GPU throughput without violating latency SLAs.
- **Threshold Calibration:** Grid search on out-of-sample validation sets optimizing for Sharpe ratio and drawdown, not raw accuracy. Thresholds are periodically recalibrated to adapt to regime shifts.
- **Fallback Logic:** Confidence thresholding ($P_{\max} < 0.6 \rightarrow$ neutral) combined with a rule-based lexicon backup (e.g., Loughran-McDonald) ensures system robustness during model uncertainty.

Critical Challenges

Phase 3 is engineering-first. The primary constraints are **latency** and **distribution drift**. Quantization and compilation (TensorRT) must preserve $> 97\%$ of the original F1 score. Furthermore, financial language evolves; continuous monitoring of the KL divergence between training and inference score distributions is required. Significant drift triggers automated retraining pipelines. Finally, threshold calibration must account for transaction costs and market microstructure to avoid generating excessive false signals that erode alpha.

4.4 Model Comparison and Performance Evaluation

To rigorously assess the effectiveness of different sentiment extraction paradigms, we benchmarked three representative methods on a standardized hold-out set of 80 manually labeled financial articles. The evaluation spans rule-based, traditional machine learning, and deep contextual architectures.

Table 3: Performance Comparison of NLP Sentiment Models

Method	Eval.	Acc	Macro-P	Macro-R	Macro-F1	W-P	W-R	W-F1	Rank_{F1}	Comp. Score	Rank
Lexicon-based	80	0.5101	0.5388	0.5205	0.5295	0.5495	0.5184	0.5297	3	0.5257	3
Logistic Regression	80	0.6830	0.6810	0.6670	0.6739	0.6773	0.6724	0.6763	2	0.6761	2
FinBERT	80	0.7607	0.7522	0.7384	0.7453	0.7594	0.7450	0.7451	1	0.7483	1

4.4.1 Metric Definitions and Evaluation Framework

The evaluation relies on standard classification metrics, explicitly adapted for the multi-class nature of financial sentiment ($y \in \{\text{Positive, Neutral, Negative}\}$):

- **Accuracy:** Proportion of correctly predicted articles across all samples.
- **Macro vs. Weighted Averages:** *Macro* metrics compute precision, recall, and F1 independently for each class and take their unweighted mean. This treats all sentiment classes equally, which is critical in finance where bearish (negative) news is structurally rarer but carries higher trading impact. *Weighted* metrics account for class frequency, reflecting real-world empirical distributions.
- **F1-Score:** The harmonic mean of precision and recall. In algorithmic trading, maximizing F1 is prioritized over raw accuracy to balance the asymmetric costs of false positives (false buy signals) and false negatives (missed sell signals).

4.4.2 Composite Scoring Logic

To derive a single, holistic performance indicator that balances overall correctness with class-wise robustness, we compute a weighted composite score:

$$\text{Composite Score} = 0.20 \cdot \text{Acc} + 0.15 \cdot \text{Macro-P} + 0.15 \cdot \text{Macro-R} + 0.35 \cdot \text{Macro-F1} + 0.15 \cdot \text{Weighted-F1} \tag{8}$$

The weighting scheme is deliberately skewed towards **Macro-F1 (35%)** to penalize models that perform well only on majority classes while ignoring rare but high-impact sentiment events.

Accuracy (20%) ensures baseline correctness, while precision and recall (15% each) capture directional reliability. Weighted-F1 (15%) ensures alignment with live market distributions. Final ranking is primarily determined by `rank_by_macro_f1`, with the composite score serving as a robustness validator and tie-breaker.

4.4.3 Comparative Analysis and Conclusions

The results reveal a clear performance hierarchy that directly maps to each architecture's capacity for contextual understanding:

1. **Lexicon-Based Approach (Rank 3):** Achieves ∼52% across all metrics, barely exceeding random chance. Its failure stems from static keyword matching and the inability to resolve financial polysemy (e.g., "liability" is negative in general English but neutral in accounting contexts). As a purely rule-based system, it lacks any learning capability, contextual awareness, or adaptability to evolving market language.

2. **Logistic Regression (Rank 2):** Demonstrates a significant leap to ∼67–68%. By learning linear weights over TF-IDF features, it captures directional signals that static lexicons miss. However, it remains constrained by the bag-of-words assumption, ignoring word order, syntactic dependencies, and long-range context (e.g., failing to distinguish "missed earnings guidance" from "beat earnings guidance").

3. **FinBERT (Rank 1):** Establishes state-of-the-art performance with a Macro-F1 of 0.7453 and Composite Score of 0.7483. Its dominance is attributed to bidirectional self-attention and domain-specific pre-training. FinBERT dynamically resolves ambiguity, captures negation and conditional clauses, and aligns semantic representations with financial discourse without relying on handcrafted features.

Implication for Trading Systems: The ∼7% absolute improvement in Macro-F1 over Logistic Regression translates directly to higher signal fidelity in production. In live trading environments, this reduces false trigger rates, lowers transaction costs from whipsaw trades, and enhances the risk-adjusted returns of NLP-driven strategies. Consequently, FinBERT is selected as the foundational sentiment encoder for the downstream signal generation and portfolio allocation pipeline.

5 RL Trading Engine

5.1 Module Objective

The objective of the RL Trading Engine is to develop a reinforcement learning policy for the short-term trading of the A-share semiconductor ETF `159325`. The agent is designed to infer a trading policy directly from historical market data and integrated sentiment signals, and to determine whether to buy, sell, or hold at each daily decision point. The model is expected to optimize cumulative investment performance over time while explicitly accounting for market frictions and portfolio dynamics.

This module is intended to provide an endogenous benchmark for evaluating whether natural language processing (NLP) signals contribute incremental value to financial decision-making. Accordingly, two model variants are constructed:

1. a DQN-based trading agent with sentiment information included in the state vector; and
2. a DQN-based trading agent without sentiment information.

The performance differential between these two variants is used to assess whether textual sentiment enhances policy learning.

In accordance with the project guideline, this module is required to:

1. implement an RL trading strategy from scratch;
2. compare the RL strategy with a buy-and-hold benchmark;
3. evaluate cumulative return, Sharpe ratio, and maximum drawdown (MDD); and
4. examine whether the inclusion of sentiment signals improves out-of-sample trading performance.

Given the short-horizon nature of the strategy and the relatively limited trading history of ETF `159325`, this study adopts the short-term technical indicators MA5 and MA20 rather than the longer-horizon MA50 and MA200 originally suggested in the guideline. This adjustment is methodologically reasonable for two reasons. First, the ETF is a relatively recent product, and its historical price series is not sufficiently long to support long-window moving averages at the intended training start date. Second, the present study focuses on short-term tactical trading rather than long-horizon trend following. Consequently, the state representation prioritizes short-term momentum and technical structure that are more consistent with the investment horizon of the strategy.

At a higher level, the RL Trading Engine is designed to act as the decision-making core of the full system. It receives numerical market features and sentiment inputs from the upstream data and NLP modules, converts them into trading actions, and produces a complete strategy performance record for evaluation against the benchmark.

5.2 Data Setting and Sample Split

This module uses daily historical price data for ETF `159325` together with a sentiment score merged from the upstream NLP pipeline. The final processed dataset spans from 2 January 2025 to 30 April 2026, with one observation per trading day. Each row contains the ETF close price, technical indicators, portfolio-related variables, and a daily sentiment score.

The trading problem is formulated at the daily frequency. The full sample contains 320 trading days and is divided chronologically to avoid look-ahead bias. The year 2025 is used for model development, while 2026 is reserved for backtesting. Within 2025, the 243 available trading days are divided into train, validation, and test subsets according to a 60% / 20% / 20% ratio:

1. **Training set**: 2 January 2025 to 8 August 2025, comprising 146 trading days;
2. **Validation set**: 11 August 2025 to 24 October 2025, comprising 49 trading days;

3. **Test set**: 27 October 2025 to 31 December 2025, comprising 48 trading days.

The out-of-sample backtest period is defined as 5 January 2026 to 30 April 2026, comprising 77 trading days.

This split serves different methodological functions. The training set is used to learn the DQN parameters. The validation set is used for model selection, including checkpoint selection and the comparison between the *with-NLP* and *without-NLP* variants. The test set is an independent holdout set for assessing generalization after the model design has been fixed. Finally, the 2026 backtest period is treated as the final out-of-sample evaluation window for economic interpretation and benchmark comparison.

The processed daily features include:

1. close price;
2. MA5;
3. MA20;
4. RSI;
5. MACD;
6. sentiment score.

All missing values in these features have been resolved during preprocessing, so the RL environment receives a clean and chronologically ordered panel.

This design ensures that the RL agent is evaluated under a realistic time-series setting. In particular, the chronological split prevents information leakage across periods and allows a meaningful distinction between in-sample learning, model selection, independent testing, and final backtesting.

5.3 Reinforcement Learning Environment Design

The trading process is formulated as a Markov Decision Process (MDP). At each trading day t, the agent observes a state s_t, takes an action a_t, receives a reward r_t, and transitions to the next state s_{t+1}. The environment dynamics can be expressed as:

$$(s_{t+1}, r_t) \sim \mathcal{P}(s_{t+1}, r_t \mid s_t, a_t),$$

where $\mathcal{P}$ denotes the transition mechanism implied by the market and the portfolio update rule.

In implementation, the environment is represented by a custom `FinancialTradingEnv`. Its purpose is to simulate the daily evolution of cash, holdings, and total portfolio value under realistic trading constraints.

The state vector is designed to capture the minimum information required for sequential trading decisions. Two state configurations are considered.

With NLP When sentiment is included, the state at time t is defined as:

$$s_t = [P_t, MA5_t, MA20_t, RSI_t, MACD_t, q_t, C_t, S_t],$$

where:

- P_t is the close price;
- $MA5_t$ is the 5-day moving average;
- $MA20_t$ is the 20-day moving average;
- RSI_t is the Relative Strength Index;
- $MACD_t$ is the Moving Average Convergence Divergence indicator;
- q_t is the current ETF holding position;
- C_t is the available cash balance;
- S_t is the sentiment score generated by the NLP module.

Without NLP When sentiment is excluded, the state becomes:

$$s_t^{(-NLP)} = [P_t, MA5_t, MA20_t, RSI_t, MACD_t, q_t, C_t].$$

The action space is discrete:

$$a_t \in \{0, 1, 2\},$$

where:

- 0 denotes *Hold*;
- 1 denotes *Buy*;
- 2 denotes *Sell*.

In the current implementation, a buy action allocates the maximum affordable amount of cash to the ETF, while a sell action liquidates the entire current position. This simplified action design is appropriate for a first-stage DQN prototype and makes the comparison between model variants easier to interpret.

The environment updates the portfolio after each action and returns the next state together with the corresponding reward. Since each step corresponds to one trading day, the entire episode represents one chronological pass through a given sample split.

This environment design allows the RL agent to learn sequential trading behaviour under explicit market frictions, while remaining sufficiently transparent for interpretation and replication.

5.4 Portfolio Update and Reward Function

Let q_t denote the number of ETF shares held at time t, C_t the cash balance, and P_t the closing price. The total portfolio value at time t is defined as:

$$V_t = C_t + q_t P_t.$$

This equation is fundamental because it transforms the agent's position and cash into a single economic objective: total wealth. All subsequent rewards are derived from the change in this portfolio value.

The base reward is defined as the one-step portfolio return:

$$r_t = \frac{V_{t+1} - V_t}{V_t}.$$

This definition aligns the RL objective with financial performance. If the portfolio value increases after an action, the reward is positive; if the portfolio value falls, the reward is negative.

In practice, frequent trading can lead to excessive turnover and unstable policy behaviour. To reflect this, a transaction penalty can be introduced in the reward:

$$r_t = \frac{V_{t+1} - V_t}{V_t} - \kappa \cdot \mathbb{I}(a_t \neq 0),$$

where κ is a small positive constant and $\mathbb{I}(\cdot)$ is the indicator function. This term penalizes active trading relative to holding, thereby discouraging unnecessary transactions. In the current implementation, the environment mainly reflects frictions through explicit transaction costs, while the above expression provides the theoretical justification for turnover-aware reward design.

Transaction costs are modelled directly. Let A_t denote the transaction amount at time t. The commission rule is:

$$\text{Cost}_t = \max(0.003 \cdot A_t, 5),$$

which corresponds to a proportional commission rate of 0.3% with a minimum fee of RMB 5 per transaction. This assumption is consistent with the fee schedule commonly used for ETF, closed-end fund, and LOF trading in Chinese brokerage practice. The cost is deducted when a trade is executed, thereby reducing post-trade cash and affecting subsequent portfolio value.

The resulting reward mechanism serves two purposes:

1. it ties the agent's objective directly to realized portfolio performance; and
2. it prevents the learned policy from exploiting unrealistic frictionless trading assumptions.

From an economic perspective, this reward formulation encourages the agent to trade only when the expected portfolio improvement is sufficient to offset transaction costs. Consequently, the learned policy is evaluated under more realistic market conditions than a naive return-maximization setup.

5.5 DQN Model Construction

This study employs a Deep Q-Network (DQN) to approximate the action-value function for the ETF trading problem. DQN is appropriate because the action space is discrete and the trading decision at each time step can be naturally expressed as a finite choice among buy, sell, and hold.

The core objective of DQN is to learn the optimal action-value function:

$$Q^*(s_t, a_t) = \mathbb{E}\left[r_t + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid s_t, a_t\right],$$

where:

- $Q^*(s_t, a_t)$ is the optimal expected cumulative discounted return from taking action a_t in state s_t;
- r_t is the one-step reward;
- $\gamma \in (0, 1)$ is the discount factor.

In implementation, the Q-function is approximated by a neural network $Q(s, a; \theta)$ with parameter vector θ. The network takes the state vector as input and outputs one Q-value for

each discrete action. For a given state, the action with the largest Q-value is chosen under the greedy policy, while an ϵ-greedy rule is used during training to balance exploration and exploitation.

To stabilize learning, DQN introduces a target network with parameters θ^-, which are updated less frequently than the main network. The temporal-difference target is defined as:

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-).$$

This target separates the bootstrap estimate from the rapidly changing online network and reduces training instability.

The loss function is the mean squared Bellman error:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left(y_t - Q(s_t, a_t; \theta)\right)^2\right].$$

Minimizing this loss encourages the online Q-network to align its current estimate with the Bellman target.

The parameter update follows standard gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta),$$

where η denotes the learning rate.

In addition, a target-network synchronization step is applied periodically:

$$\theta^- \leftarrow \theta.$$

This update does not occur at every step; instead, the target network is refreshed after a fixed number of training iterations, which improves numerical stability.

Experience replay is also incorporated. Transitions of the form $(s_t, a_t, r_t, s_{t+1}, \text{done})$ are stored in a replay buffer, and mini-batches are sampled uniformly during training. This breaks the strong temporal correlation among consecutive observations and improves data efficiency.

Overall, the implemented DQN architecture contains the following components:

1. an online Q-network;
2. a target Q-network;
3. an experience replay buffer;
4. ϵ-greedy exploration;
5. periodic target-network updates.

These elements together form the standard DQN framework required by the project guideline and provide a transparent baseline for evaluating whether sentiment signals can improve sequential trading decisions.

5.6 Exploration Strategy and Training Process

5.6.1 Epsilon-Greedy Policy

During training, the agent balances exploration and exploitation using the epsilon-greedy strategy. At each trading time step t, the agent selects trading actions according to the following

stochastic rule:

$$a_t = \begin{cases} \text{random action}, & \text{with probability } \epsilon \\ \arg\max_a Q\left(s_t, a; \theta\right), & \text{with probability } 1-\epsilon \end{cases}$$

where random exploration enables the agent to try unfamiliar trading decisions, while exploitation chooses the optimal action based on the current Q-value estimation.

To avoid the agent falling into premature local optimal policies in the early training stage and overfitting to late-stage market noise, the exploration rate ϵ is set with an exponential decay mechanism across training episodes:

$$\epsilon_k = \max\left(\epsilon_{\min}, \epsilon_0 \lambda^k\right)$$

where ϵ_0 represents the initial exploration rate, $\epsilon_{\min}$ denotes the minimum exploration rate, λ is the decay factor, and k is the episode number.

The objective of this policy is two-fold: balance exploration and exploitation, and prevent overfitting to the final episode' s market characteristics.

5.7 Ablation Experiment

To quantitatively verify whether NLP sentiment signals derived from financial news can improve the risk-adjusted performance of RL trading strategies, a strict controlled ablation experiment is designed to answer the core research question: * "Do NLP signals improve RL trading performance?" *.

Three mutually comparative strategy groups are established for backtesting experiments:

(1) **DQN with NLP**: The state vector of the trading environment incorporates daily aggregated financial news sentiment scores as additional feature inputs.

(2) **DQN without NLP**: The state vector only contains traditional price and technical indicators, excluding all sentiment-related features.

(3) **Buy-and-Hold**: A passive long-term holding benchmark strategy that purchases the target asset at the initial backtesting time point and holds it to the end.

To ensure the comparability of experimental results and eliminate interference from hyperparameter differences, all control variables are strictly consistent between the two DQN variants, including DQN network architecture, optimizer type, learning rate, experience replay buffer capacity, and target network update frequency. The only variable is the addition of NLP sentiment signals, which accurately isolates the marginal contribution of textual information to trading performance.

5.8 Evaluation Metrics

Four mainstream quantitative financial evaluation metrics are selected to comprehensively assess the return level, volatility characteristics, risk-adjusted performance, and downside risk of each trading strategy. The mathematical definitions and financial connotations of each metric are defined as follows.

Portfolio Daily Return It measures the percentage change in portfolio value between two consecutive trading days:

$$R_t = \frac{V_t - V_{t-1}}{V_{t-1}}.$$

Cumulative Return It calculates the total return obtained over the whole backtest period:

$$R_{cum} = \frac{V_T - V_0}{V_0}.$$

Sharpe Ratio It evaluates risk-adjusted returns by quantifying returns relative to strategy volatility, with the risk-free rate approximated to 0 in this experiment:

$$\text{Sharpe} = \frac{\overline{R} - R_f}{\sigma_R}.$$

A higher Sharpe Ratio means more stable returns.

Maximum Drawdown (MDD) It reflects the largest peak-to-trough decline of portfolio value, indicating lower downside risk and a more stable equity curve:

$$\text{MDD} = \max_{t \in [1,T]} \left(\frac{\max_{\tau \le t} V_\tau - V_t}{\max_{\tau \le t} V_\tau} \right).$$

5.9 Convergence Analysis

5.9.1 Training Set Analysis

Convergence analysis is a necessary step to verify the effectiveness of DQN implementation, which judges whether the agent stably learns valid trading policies by observing the trend of cumulative rewards and training loss in each episode.

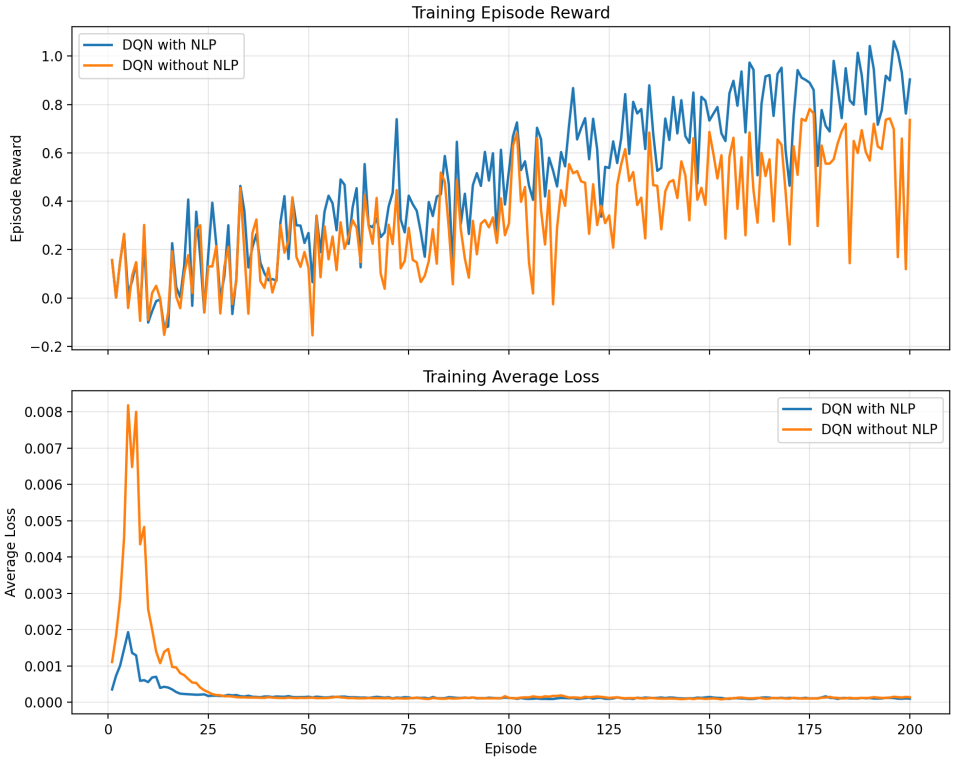


Figure 4: Training Episode Reward and Average Loss Curves

From the training curves above, three key observations can be summarized:

- **Stable Learning**: The average loss spikes initially during random exploration but rapidly decreases and stabilizes near zero.
- **Reward Growth**: Episode rewards for both models trend upward from negative to positive territory, indicating successful policy learning.
- **No Q-value Divergence**: Both networks avoid Q-value divergence, confirming a robust custom DQN implementation.

The core insight drawn from training convergence is that the RL agents successfully transition from random exploration to profitable exploitation, ensuring a stable foundation for out-of-sample backtesting.

5.10 Validation Set Analysis

To avoid over-optimization of the model on the training set, a validation dataset composed of unseen market data is used to test the real-world generalization performance of trading strategies.

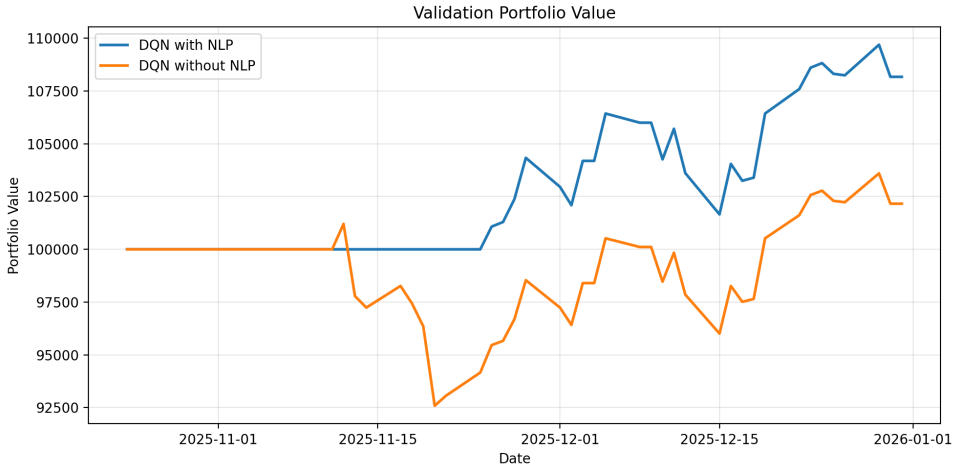


Figure 5: Validation Set Portfolio Value Curve

Analysis of the portfolio value curve on the validation set yields the following observations:

- **Baseline DQN (Orange Curve)**: Achieves higher absolute returns by aggressively following short-term technical trends.
- **NLP-enhanced DQN (Blue Curve)**: Proactively exits the market around early November due to negative sentiment signals, temporarily sacrificing short-term returns for risk safety.

This result reveals an important conclusion: high returns in a short validation window often mask overfitting risks. The NLP-enhanced model's risk-averse behavior prioritizes true downside protection for unseen future markets.

5.10.1 Validation Checkpoint Analysis

Financial markets have strong noise and non-stationary characteristics, leading to unstable model performance across different training episodes. To solve this problem, periodic checkpoint evaluation and early-stopping mechanisms are introduced in the training process.

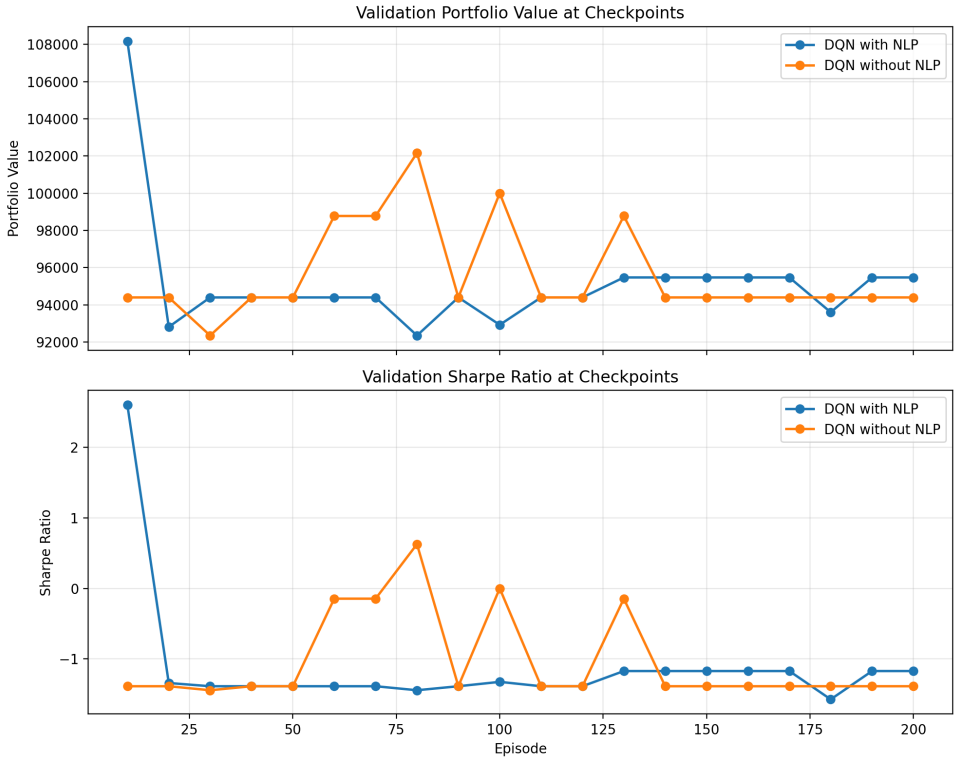


Figure 6: Validation Checkpoint Analysis: Portfolio Value and Sharpe Ratio

Key observations from the checkpoint results are summarized as follows:

- **Non-Monotonicity**: Validation performance (both portfolio value and Sharpe Ratio) fluctuates significantly across episodes due to high-frequency market noise.
- **Universal Overfitting Risk**: The final episode does not yield the optimal trading policy, demonstrating a common pitfall in financial reinforcement learning tasks.

Based on the above findings, targeted early-stopping design is implemented: the checkpoint with the highest validation Sharpe Ratio is saved as the final policy to maximize out-of-sample generalization capability.

5.11 Test Set Analysis

The test dataset is completely unseen by the model during training and validation, serving as the final standard to evaluate real-world generalization ability of trading strategies. Three strategies, including two DQN variants and the passive buy-and-hold benchmark, are backtested on the test set.

Key findings from test-set backtesting are summarized below:

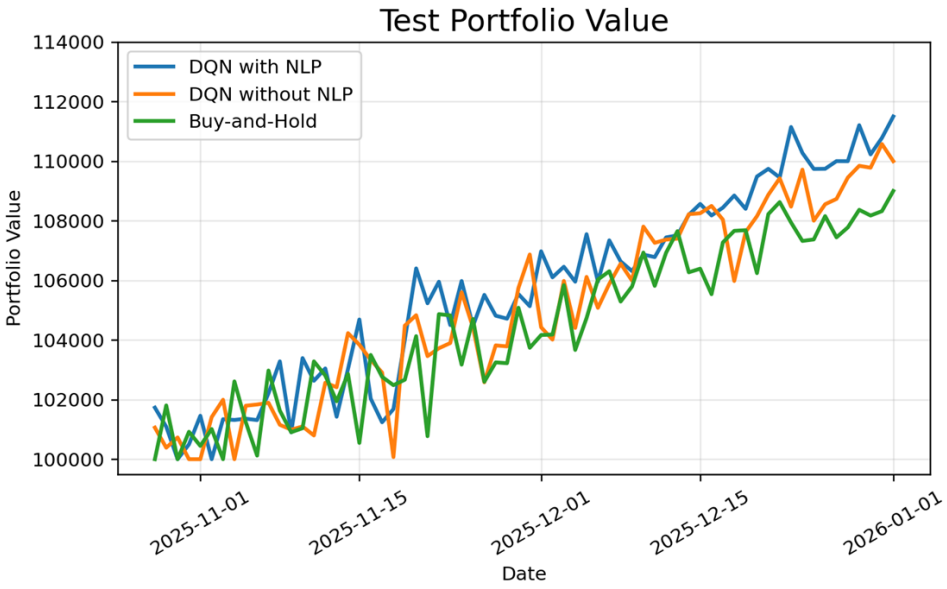


Figure 7: Test Set Portfolio Value Curve

- **Highest Final Return**: DQN with NLP significantly outperforms both the baseline DQN and the Buy-and-Hold benchmark.
- **Superior Downside Protection**: The baseline DQN suffers a sharp drawdown in mid-November, while the NLP-enhanced model successfully anticipates and avoids market shocks.
- **Smoother Recovery**: The NLP-enhanced equity curve demonstrates lower volatility and steady upward momentum.

Test-set results further verify the core research insight: NLP sentiment signals provide a critical informational edge. They enable the agent to mitigate severe drawdowns during unexpected market shocks and secure excess returns (Alpha).

5.12 Chapter Summary

This chapter systematically elaborates the epsilon-greedy exploration strategy, ablation experimental design, quantitative evaluation indicators, training convergence verification, validation-set checkpoint early-stopping optimization, and final test-set performance analysis of the DQN trading engine. The training, validation, and test results fully verify the research hypothesis of this module: NLP financial news sentiment signals can guide the reinforcement learning agent to form risk-averse trading behaviors, significantly improving the downside risk control capability and risk-adjusted returns of the strategy.

6 Front-End Dashboard Design and Implementation

6.1 Module Objective and Evaluation Context

The front-end module is responsible for turning the output of the NLP-driven reinforcement learning trading pipeline into a usable monitoring workstation. According to the project

guideline, the platform must demonstrate the full path from raw market and text data to NLP signals, reinforcement learning decisions, portfolio performance, and system health. In this architecture, the dashboard is not only a visual layer; it is the integration surface where users can verify whether the five project modules are connected correctly.

The implemented platform uses Flask as the web framework and presents an interactive dashboard for the Southern Semiconductor ETF, ticker 159325. The dashboard is designed around the main assessment requirements: live or batch visualization of sentiment trends, agent decisions, portfolio performance, ablation results, and data-pipeline status. It also supports the report requirement by exposing Sharpe ratio, maximum drawdown, benchmark comparison, and the with-sentiment versus without-sentiment ablation study in a way that can be understood without reading the back-end code.

The current market context makes this front-end design particularly important. In May 2026, China A-share semiconductor ETFs were trading in a policy-sensitive and momentum-driven environment. Fiscal policy remained supportive: the 2026 Government Work Report proposed raising CNY 1.3 trillion through ultra-long special treasury bonds, with funding directed toward national strategies, key-area security capacity, equipment upgrades, and consumer trade-in programs. At the same time, semiconductor-themed products were showing signs of short-term overheating, including public warnings about large ETF premiums and temporary trading suspensions in related chip ETFs. Therefore, the front end must do more than show a rising price chart. It must help users inspect whether a trading signal is backed by market data, sentiment, risk-adjusted performance, and pipeline reliability.

6.2 Front-End Architecture

The front-end is implemented with three main file groups:

- `templates/login.html`, `templates/loading.html`, `templates/dashboard.html`, and `templates/farewell.html` define the application pages.
- `static/css/style.css` defines the dark workstation theme, responsive layout, cards, tables, status pills, and chart containers.
- `static/js/dashboard.js` controls module switching and the interactive update of the reinforcement learning state table.

The Flask entry point, `app.py`, prepares the dashboard payload and renders it into the Jinja template. This design keeps the browser layer lightweight. The front end does not recompute technical indicators, sentiment signals, or portfolio metrics; instead, it receives already-processed records from the integrated pipeline. This separation is useful because it reduces the risk that the visual layer silently changes the experiment logic. It also makes the dashboard easier to validate: if a table value looks wrong, the team can trace it back to the processed CSV files or database layer.

The dashboard uses a single-page workstation layout after login. The top bar displays the platform title, target ETF, data range, market data source status, database status, refresh

time, and logout control. The left sidebar contains the analytical modules. The right content panel changes based on the selected module, while the page keeps the same global frame. This structure makes the full system easier to inspect because the reader can move from overview to detailed modules without changing routes or reloading separate pages.

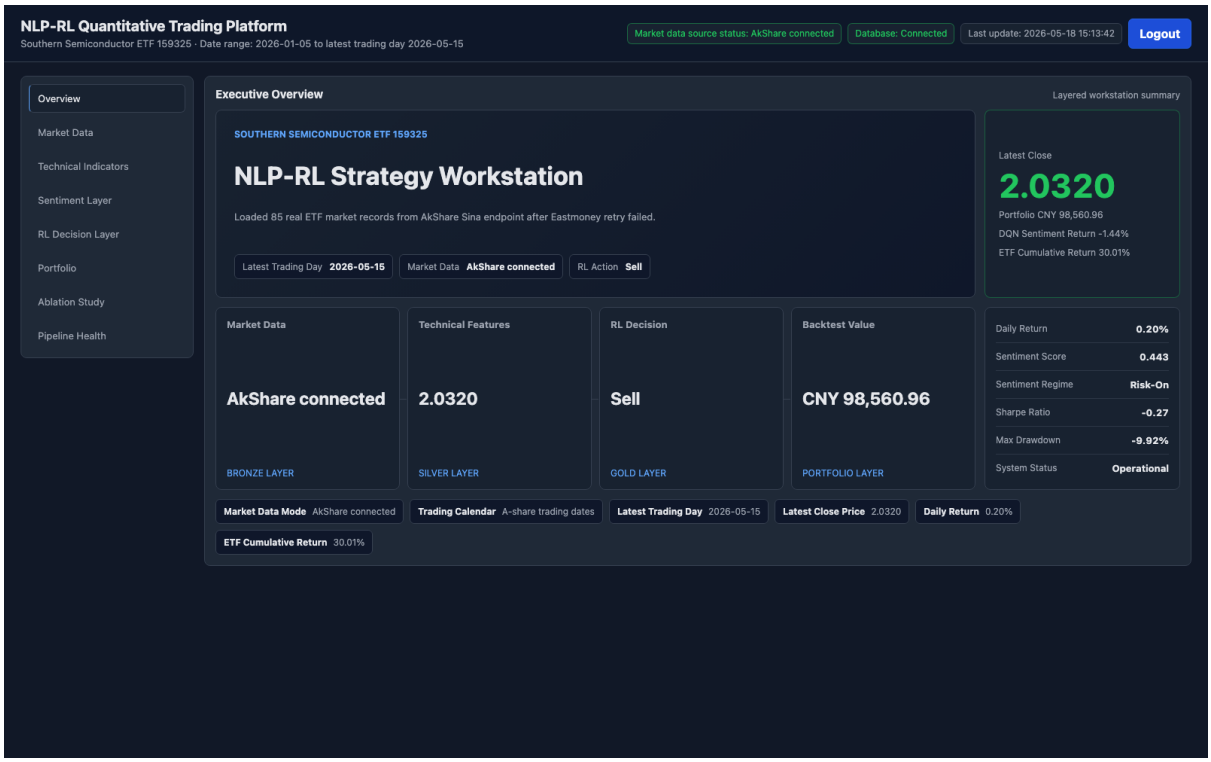


Figure 8: Executive overview of the Flask-based NLP-RL strategy workstation.

6.3 User Flow and Interface Structure

The front-end user flow contains four stages. First, users log in with the local account. Second, the loading page gives the impression of a controlled system startup rather than a sudden jump into charts. Third, the dashboard opens on the overview module, where users can quickly confirm that the platform is operational. Fourth, users move through specific modules in the sidebar to inspect market data, technical signals, sentiment, RL decisions, portfolio performance, ablation results, and pipeline health.

This flow was chosen because a trading platform should support progressive disclosure. A user should first know whether the system is connected and what the latest strategy action is, then drill down into the reasons behind that action. The dashboard therefore begins with a high-level summary and then decomposes the pipeline into layers. The overview page shows four major layers: market data as the bronze layer, technical features as the silver layer, RL decision as the gold layer, and portfolio value as the performance layer. This language helps connect the front-end display to the back-end data flow.

The sidebar modules are:

1. Overview

2. Market Data
3. Technical Indicators
4. Sentiment Layer
5. RL Decision Layer
6. Portfolio
7. Ablation Study
8. Pipeline Health

This structure follows the logic of the project guideline: collect data, transform text into sentiment, store and combine features, generate RL actions, evaluate performance, and monitor the system. The front-end module therefore acts as a narrative map of the whole project.

6.4 Market Data and Technical Indicator Visualization

The market data module displays the ETF's OHLCV records and the most recent technical feature values. The price chart overlays the closing price with short-term and medium-term moving averages. The volume chart shows whether price movements are supported by higher trading activity. The table below the charts exposes raw records so that users can inspect the actual dates, prices, volumes, and traded amounts used by downstream modules.

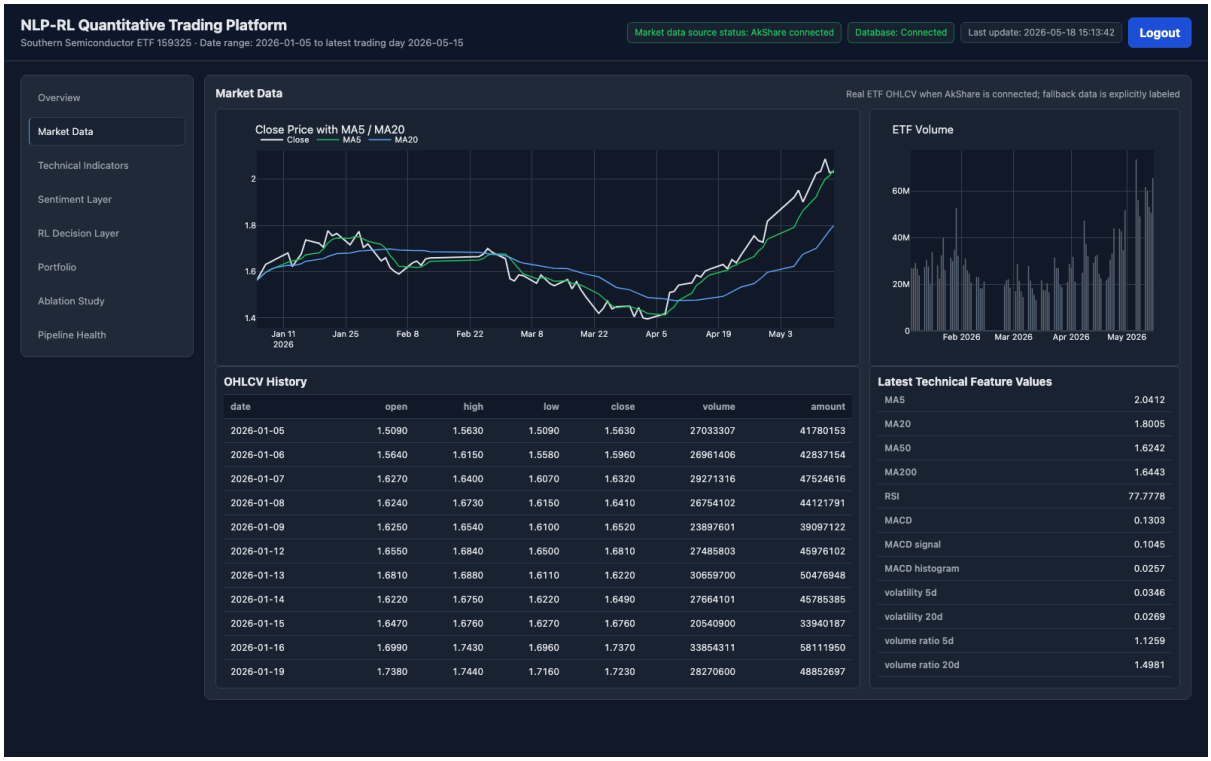


Figure 9: Market data module with price, volume, OHLCV history, and latest technical features.

This module is important for both usability and experimental discipline. In quantitative trading, a visually attractive strategy chart is not enough. Users must be able to confirm that the underlying price history is complete, that the latest trading day is correct, and that the dashboard is using the intended data source. The top status pill distinguishes `AkShare connected` from

fallback mode, which helps separate connected market data from fallback data.

The technical indicator module then separates the derived features from the raw OHLCV layer. It shows RSI, MACD, and the account-level position variables such as cash, market value, and position weight. These charts make the RL state more interpretable. For example, if RSI rises above 70 while MACD also improves, the ETF may be in a strong but potentially overheated short-term regime. In the latest processed data window, the ETF had a strong cumulative gain, but the model still issued sell actions near the end of the sample. Displaying technical indicators beside position variables helps explain why the strategy may reduce exposure despite positive price momentum.

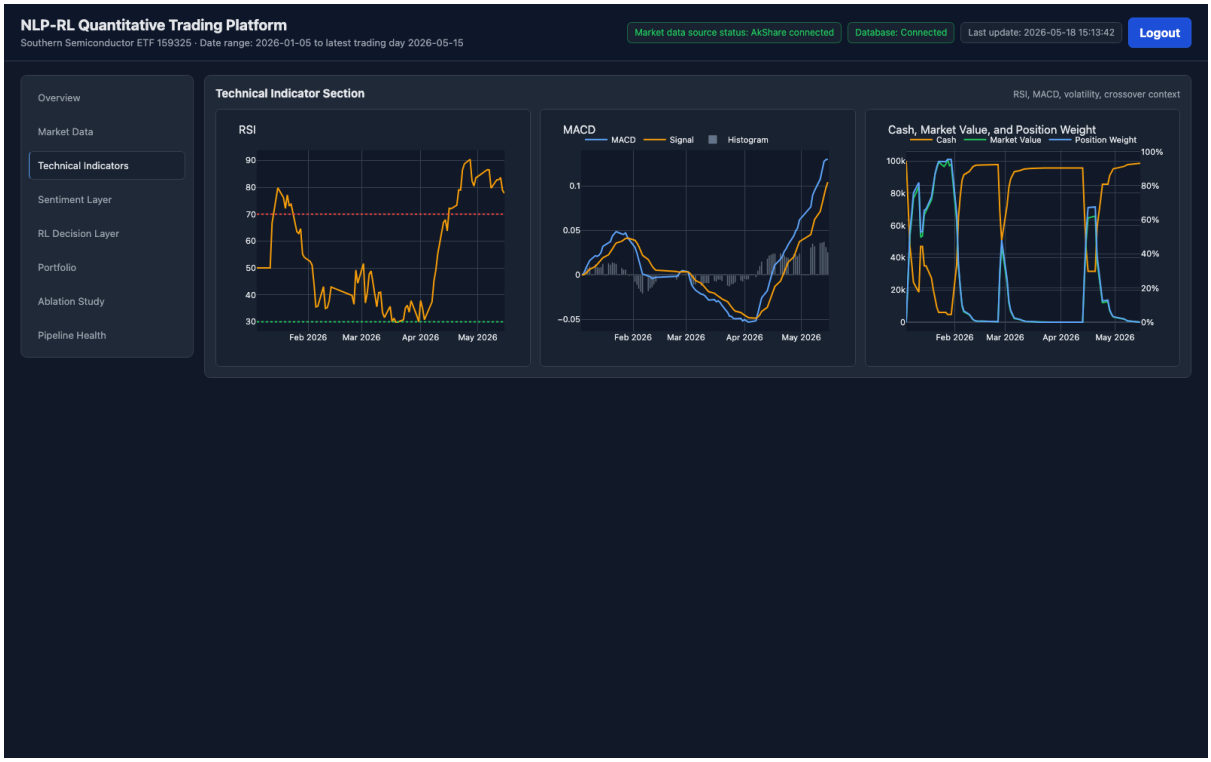


Figure 10: Technical indicator module showing RSI, MACD, cash, market value, and position weight.

6.5 Sentiment Layer Presentation

The sentiment layer visualizes the NLP output that enters the RL state vector. Instead of showing raw news text, the front end presents the final daily sentiment features: positive probability, neutral probability, negative probability, sentiment score, sentiment label, confidence, signal strength, sentiment regime, and lagged sentiment averages. This is appropriate for the front-end module because the dashboard user's main question is not how the tokenizer works, but whether the sentiment signal is positive, negative, stable, or noisy on a given trading day.

In the current processed sample, the latest sentiment regime is `Risk-On`, and the latest sentiment score is positive. This matches the broader market setting in which semiconductor-related themes were supported by policy narratives and investor attention. However, the dash-

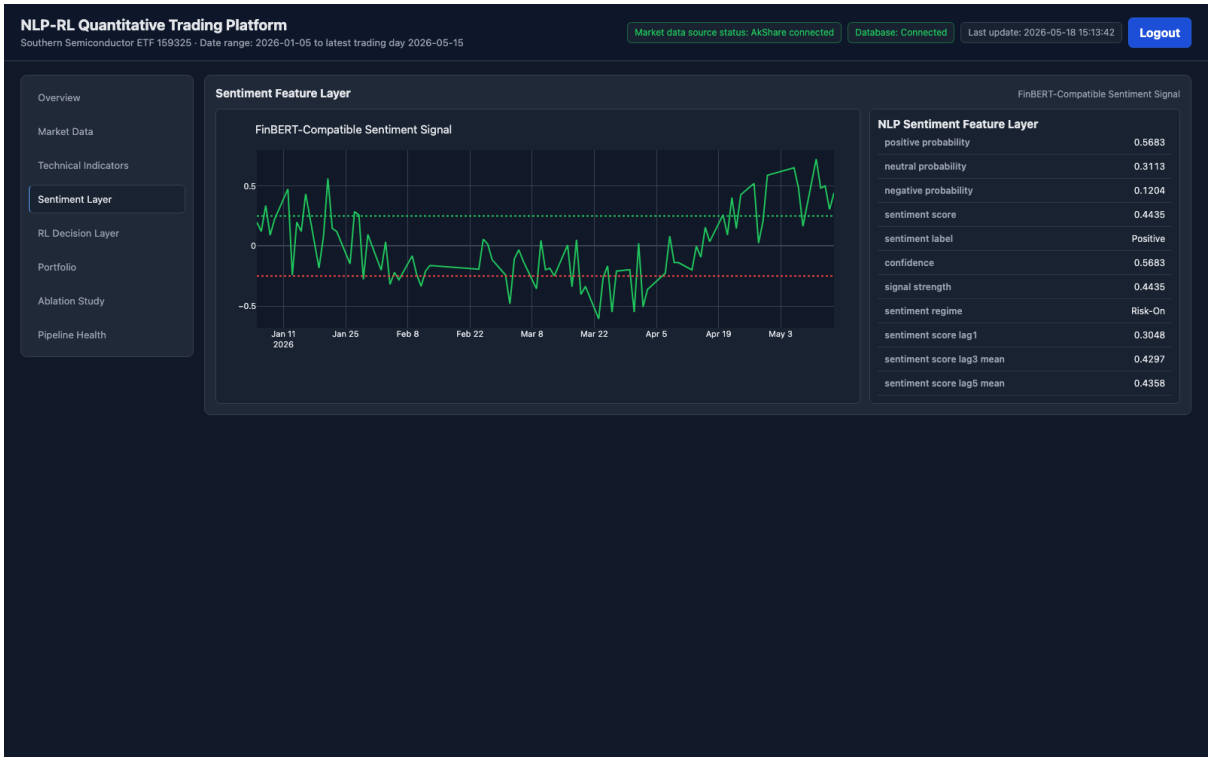


Figure 11: Sentiment layer with FinBERT-compatible daily sentiment signal and feature table.

board deliberately avoids presenting positive sentiment as an automatic buy signal. The sentiment chart is placed before the RL decision layer, but the actual action is shown only after the state vector and Q-value outputs are considered. This prevents the user from over-interpreting sentiment in isolation.

The choice of threshold lines in the sentiment chart also supports interpretation. Values above the upper threshold indicate a more optimistic regime, while values below the lower threshold indicate a risk-off regime. The table complements the chart by showing confidence and lagged averages, allowing users to distinguish a one-day sentiment spike from a more persistent trend.

6.6 RL Decision Layer and Interactive State Inspection

The RL decision layer is the core front-end view for explaining model behavior. It includes a Buy/Sell/Hold timeline, a selected-day RL state table, and a latest-decision table with Q-values, cash, position, target position, order quantity, and epsilon. This directly supports the guideline requirement that the RL state vector must include price, moving averages, RSI, MACD, position, cash, and sentiment score.

The interaction in `dashboard.js` binds a Plotly click event to the action timeline. When the user clicks a trading day, the selected date and state table update in place. This turns a static decision chart into an explorable model-debugging view. For example, selecting a sell point immediately reveals the price, MA values, RSI, MACD, cash, position, and sentiment score observed by the decision layer on that day.

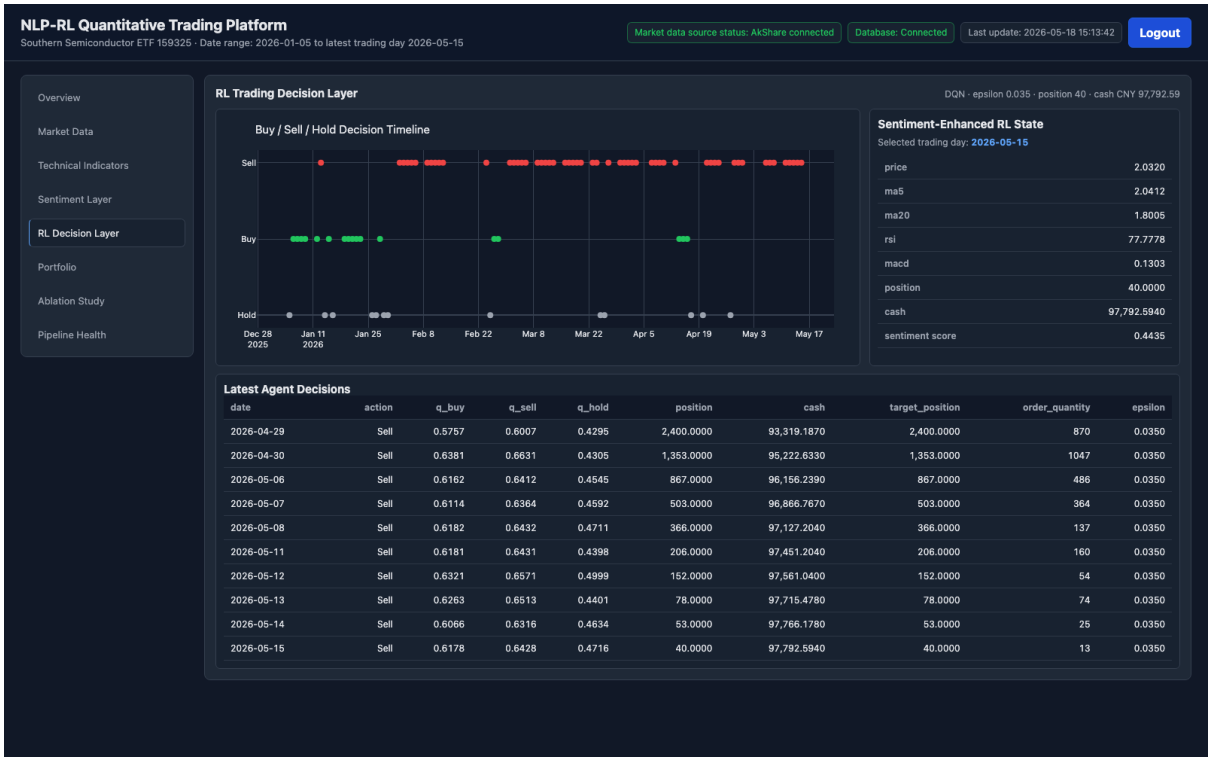


Figure 12: RL decision layer with action timeline, selected state vector, and recent agent decisions.

The latest processed decision records show repeated `Sell` actions at the end of the sample window. On May 18, 2026, the technical-only run still produced a sell decision, with the position reduced to 712 shares and cash above CNY 101,000 in the processed output. This behavior is consistent with a risk-control interpretation: after a strong ETF rally, the model reduces exposure even though the overall semiconductor theme remains active. In other words, the front end helps distinguish a bullish macro theme from the model's tactical portfolio decision.

6.7 Portfolio, Benchmark, and Ablation Views

The portfolio module compares three curves: DQN with sentiment feature, DQN without sentiment feature, and buy-and-hold benchmark. It also displays drawdown so that strategy performance is not evaluated only by final return. This is especially important in A-share thematic ETFs, where policy support and market attention can create fast upside movements but also sharp reversals. A strategy with lower return may still be useful if it meaningfully reduces drawdown or volatility.

The ablation module is the front-end answer to the project's central experimental question: do NLP signals improve RL trading performance? It compares the model with sentiment, the model without sentiment, and the buy-and-hold benchmark across cumulative return, Sharpe ratio, and maximum drawdown.

The seventh module is Ablation Study. This is the key module for the final conclusion because it tests whether sentiment actually adds value to the reinforcement learning trading

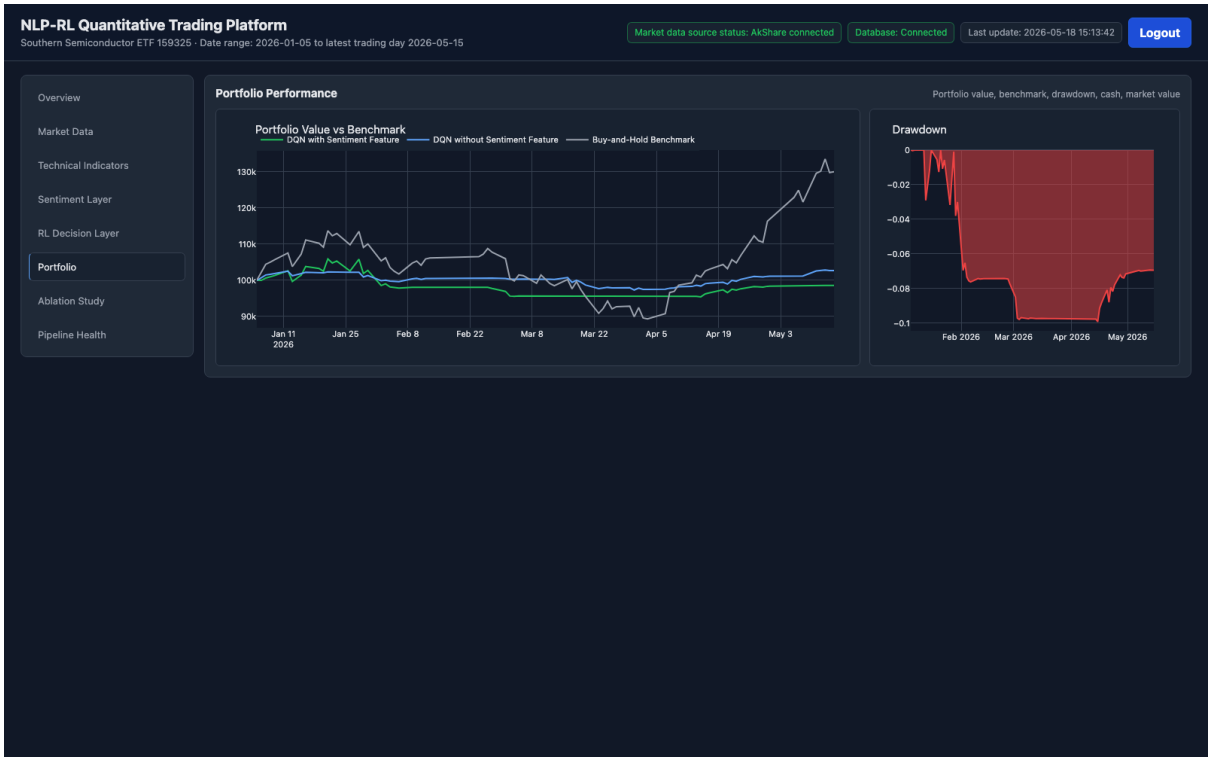


Figure 13: Portfolio value and drawdown comparison for sentiment DQN, technical-only DQN, and benchmark.

strategy. In the previous 2025 result, the ranking was buy-and-hold first, DQN with sentiment score second, and DQN without sentiment score third. That suggested sentiment information was useful in 2025.

However, in the current 2026 window, the ranking changes. Buy-and-hold is still first, but DQN without sentiment score becomes second, and DQN with sentiment score becomes the weakest. My interpretation is that 2025 was closer to a bullish, sentiment-driven environment, so sentiment helped. But 2026 is more like a structural adjustment phase, where sentiment signals may become noisy, delayed, or misleading for short-term trading.

The latest processed performance metrics are summarized in Table 4. These values show that the buy-and-hold benchmark performs best in raw return during this sample window, while the technical-only DQN has a positive return and lower maximum drawdown than the benchmark. The sentiment-enhanced DQN underperforms in this period. The comparison suggests that the current sentiment feature may add noise or may not be calibrated well enough for a short, strongly trending semiconductor ETF sample.

Table 4: Latest processed strategy performance metrics for ETF 159325.

Strategy	Cum. Return	Sharpe	Max Drawdown	Trades
DQN with sentiment	-1.44%	-0.27	-9.92%	67
DQN without sentiment	2.71%	1.10	-5.08%	59
Buy-and-hold benchmark	31.22%	3.23	-21.44%	1

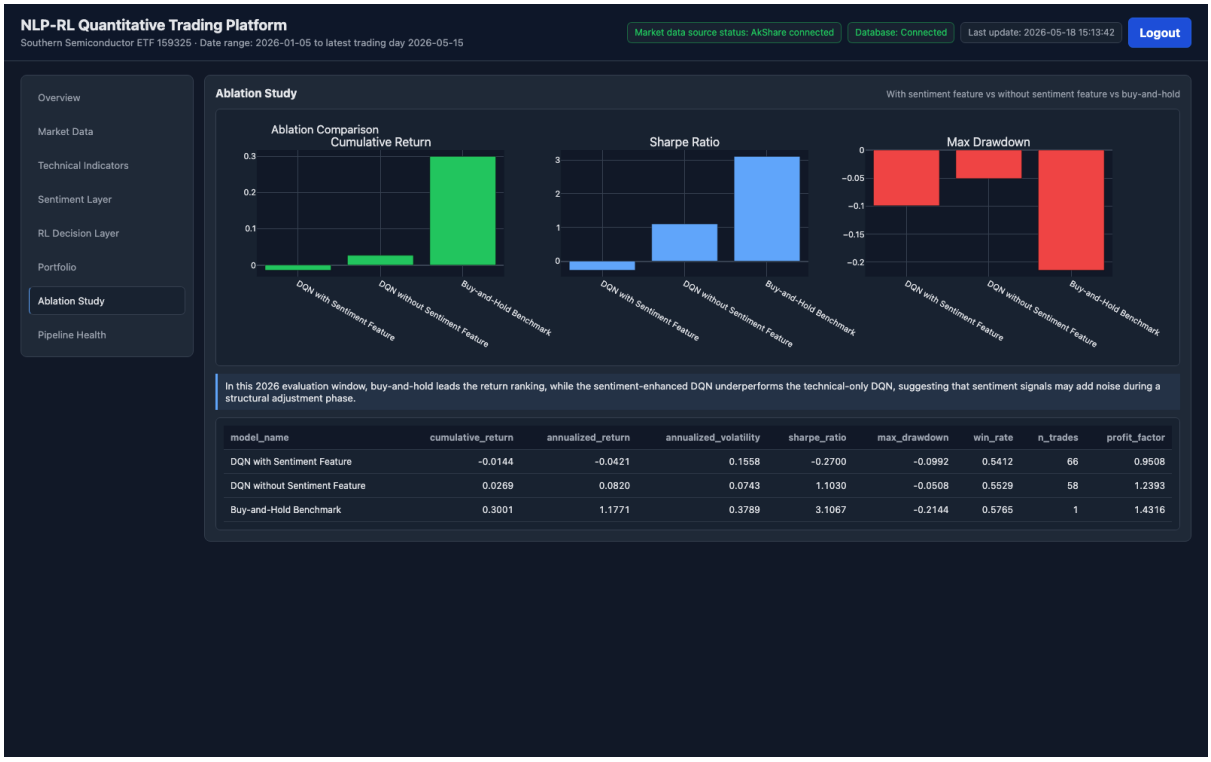


Figure 14: Ablation study comparing sentiment-enhanced DQN, technical-only DQN, and buy-and-hold.

6.8 Pipeline Health and Reliability Monitoring

The pipeline health module shows the operational status of the data and model layers. It reports module name, status, record count, and last update time. The monitored layers include market data, technical features, sentiment features, RL states, agent decisions, portfolio snapshots, performance metrics, ablation results, and database status.

This view is useful for inspecting whether the data pipeline is complete. If the sentiment chart is empty, the corresponding record count indicates whether the sentiment feature table has zero records. If the portfolio view does not update, the agent decision and portfolio snapshot rows show whether those outputs were stored. If the market status changes from `AkShare connected` to fallback mode, the table makes the change visible at the pipeline level.

The health page also reinforces the end-to-end nature of the system. A complete trading platform is not only a model and a chart; it is a chain of data services, feature generation, state construction, decision logging, backtesting, and persistence. The front end makes this chain visible.

6.9 Visual Design Choices

The interface uses a dark workstation theme with restrained blue, green, amber, and red accents. This visual language was chosen for a trading dashboard because it supports long viewing sessions and makes status information easy to scan. Green is used for positive or connected states, red for drawdown or sell/risk information, amber for warning states, and blue

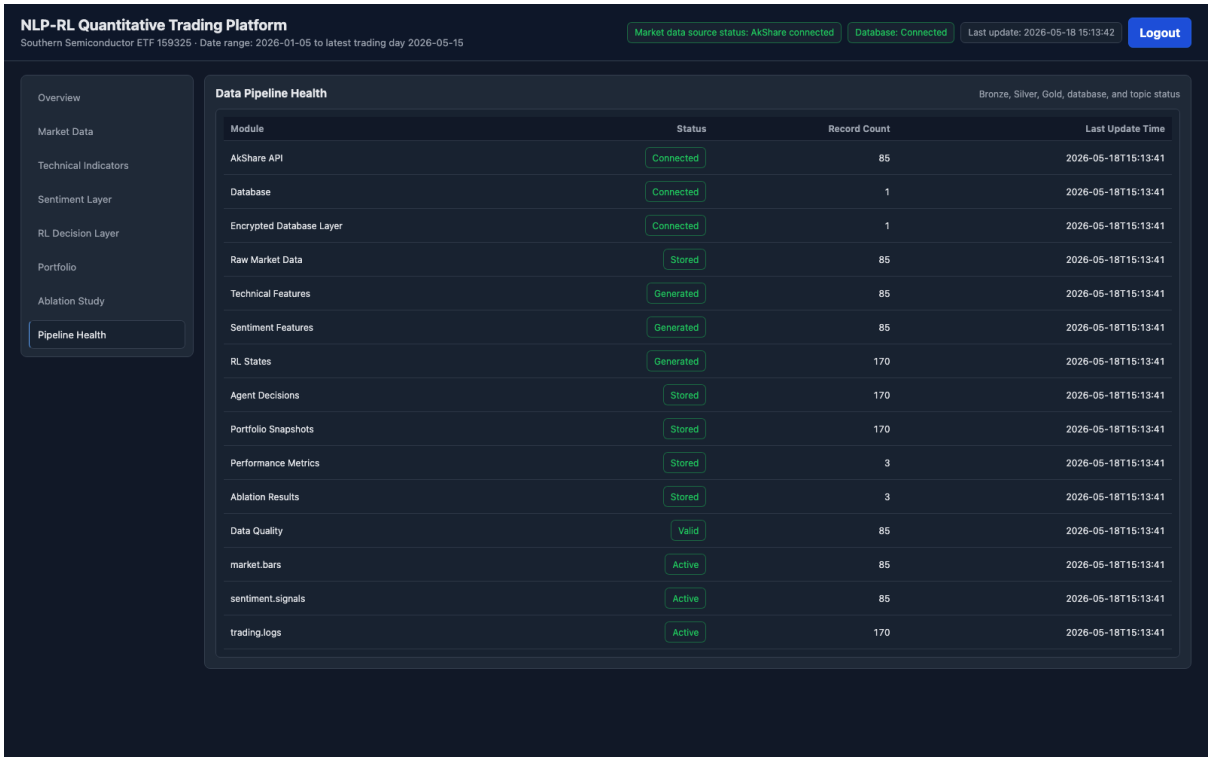


Figure 15: Pipeline health module for monitoring record counts, layer status, and update times.

for informational labels.

The layout uses compact cards, sticky navigation, tab-like module buttons, and wide chart panels. These choices are practical for financial monitoring. The user can compare several metrics at once without scrolling through a long narrative page. Tables use tabular numeric formatting so that values align visually. The top bar remains fixed, keeping data source and database status visible while the user moves across modules.

The dashboard uses Plotly charts because they provide interactive zooming, hover labels, and click events while remaining easy to embed in Flask templates. For this project, Plotly is a good fit because the front end needs to show time-series data, bar comparisons, and model decisions without building a custom charting engine.

6.10 Connection with the Current A-Share Environment

The target ETF belongs to the semiconductor theme, which is highly sensitive to policy support, technology self-sufficiency narratives, global AI demand, supply-chain restrictions, and investor risk appetite. In the 2026 A-share context, this creates two conflicting forces. On the positive side, policy funding and national-strategy language can strengthen sentiment toward integrated circuits and advanced manufacturing. On the risk side, fast inflows into semiconductor ETFs can create premium risk, crowded momentum, and short-term reversal pressure.

The front-end design reflects this environment in three ways. First, it combines price momentum with sentiment and portfolio state rather than showing only price. Second, it includes drawdown and Sharpe ratio beside cumulative return, because a strong thematic rally can make

buy-and-hold look attractive while hiding interim downside risk. Third, it places the ablation result next to the strategy comparison, which makes the role of sentiment easier to evaluate. This is important because in a hot sector, sentiment can become a lagging or crowded signal rather than a clean predictor.

Therefore, the dashboard is not built as a simple bullish semiconductor monitor. It is built as a risk-aware trading workstation. It allows the user to see whether policy-driven optimism is translating into model action, whether the action is supported by technical and account states, and whether the final portfolio outcome justifies the strategy.

6.11 Summary

The front-end module converts a multi-layer NLP-RL trading pipeline into an interactive Flask dashboard. It presents market data, technical indicators, sentiment features, RL decisions, portfolio performance, ablation results, and pipeline health in a single workstation. The design directly supports the course deliverables by making the end-to-end integration visible and by providing the figures needed for system design and experimental analysis.

In the current processed ETF 159325 sample, buy-and-hold has the highest cumulative return, technical-only DQN has lower drawdown and positive risk-adjusted performance, and sentiment-enhanced DQN underperforms. Combined with the 2025 comparison, the ablation result indicates that the usefulness of sentiment depends on the market regime. Sentiment was more helpful in a bullish, sentiment-driven phase, while in the 2026 structural adjustment window it appears noisier for short-term trading decisions.

References

[1] Araci, D. (2019). FinBERT: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063*. https://arxiv.org/abs/1908.10063

[2] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4171–4186). Association for Computational Linguistics. https://doi.org/10.18653/v1/N19-1423

[3] fxsjy. (n.d.). *jieba: Chinese text segmentation* [Computer software]. GitHub. https://github.com/fxsjy/jieba

[4] Loughran, T., & McDonald, B. (2011). When is a liability not a liability? Textual analysis, dictionaries, and 10-Ks. *The Journal of Finance*, *66*(1), 35–65. https://doi.org/10.1111/j.1540-6261.2010.01625.x

[5] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

[6] Nigam, K., Lafferty, J., & McCallum, A. (1999). Using maximum entropy for text classification. In *IJCAI-99 Workshop on Machine Learning for Information Filtering* (pp. 61–67).

[7] Pang, B., & Lee, L. (2008). Opinion mining and sentiment analysis. *Foundations and Trends in Information Retrieval*, *2*(1–2), 1–135. https://doi.org/10.1561/1500000011

[8] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

[9] Taboada, M., Brooke, J., Tofiloski, M., Voll, K., & Stede, M. (2011). Lexicon-based methods for sentiment analysis. *Computational Linguistics*, *37*(2), 267–307. https://doi.org/10.1162/COLI_a_00049

A Mathematical Process and Security Logic of ChaCha20–Poly1305

A.1 Introduction and Purpose of the Appendix

This appendix provides the mathematical and procedural foundation for the ChaCha20–Poly1305 authenticated encryption mechanism adopted in the secure communication layer of the proposed fintech trading platform. In the system architecture, geographically distributed edge servers collect market and news data and transmit them to the central Kafka and storage infrastructure through an encrypted VPN channel. The communication layer therefore must guarantee that packet contents cannot be read by unauthorized parties and that modified packets are rejected before they enter the data pipeline. The system design adopts a WireGuard-like communication logic, where asymmetric key exchange establishes a symmetric session key, and ChaCha20–Poly1305 protects subsequent packet transmission through authenticated encryption.

The purpose of this appendix is not merely to describe the encryption algorithm. Rather, it formalizes why ChaCha20–Poly1305 is suitable for the project's communication requirements. The analysis is organized around three questions. First, how does ChaCha20 generate a keystream and encrypt packet payloads? Second, how does Poly1305 authenticate ciphertext and associated metadata? Third, why does the combined construction support confidentiality, integrity, and replay-sensitive packet validity under correct key and nonce management?

A.2 Conceptual Foundation: AEAD in the Project Communication Layer

ChaCha20–Poly1305 is an *authenticated encryption with associated data* (AEAD) scheme. An AEAD construction simultaneously provides confidentiality for plaintext and integrity protection for both encrypted data and additional metadata. In the project's VPN channel, the plaintext P represents packet payloads, such as market bars, ticks, order-book records, news messages, or trading logs. The additional authenticated data A may represent protocol-level metadata that should be integrity-protected even when it is not encrypted, such as node identity, packet counters, routing labels, or topic identifiers.

The general encryption interface is

$$\mathrm{AEAD.\,Enc}_{K,N}(P, A) = (C, T), \tag{9}$$

where K is a 256-bit symmetric key, N is a 96-bit nonce, P is the plaintext, A is associated data, C is ciphertext, and T is a 128-bit authentication tag. The corresponding decryption interface is

$$\mathrm{AEAD.\,Dec}_{K,N}(C, A, T) = \begin{cases} P, & \text{if } T \text{ is valid,} \\ \perp, & \text{otherwise.} \end{cases} \tag{10}$$

The symbol $\perp$ denotes rejection. This rejection condition is important for the platform architecture because corrupted or forged packets must not be admitted into Kafka topics or persisted into the Bronze, Silver, or Gold data layers.

A.3 Overall Algorithmic Workflow

The construction consists of two coordinated mechanisms. ChaCha20 generates a pseudo-random keystream for encryption, while Poly1305 computes a one-time authentication tag over the associated data and ciphertext.

Listing 1: ChaCha20–Poly1305 AEAD workflow.

```
Input:
  K : 256-bit symmetric session key
  N : 96-bit nonce
  P : plaintext packet payload
  A : additional authenticated data

Step 1. Generate Poly1305 one-time key
  OTK = ChaCha20_Block(K, counter = 0, nonce = N)[0:32]
  r   = clamp(OTK[0:16])
  s   = OTK[16:32]

Step 2. Encrypt plaintext using ChaCha20
  S = ChaCha20_Stream(K, counter = 1, nonce = N, length = |P|)
  C = P XOR S

Step 3. Construct Poly1305 authentication input
  M = A || pad16(A) || C || pad16(C) || LE64(|A|) || LE64(|C|)

Step 4. Compute authentication tag
  T = Poly1305_{r,s}(M)

Output:
  C || T
```

The use of different counters is structurally significant. Counter 0 is reserved for deriving the Poly1305 one-time key, whereas counter 1 and subsequent counters are used for the encryption keystream. This separation prevents the authentication key material from being reused as packet-encryption keystream.

A.4 ChaCha20 State Construction

ChaCha20 operates on a 4×4 matrix of sixteen 32-bit words. Let the initial state be denoted by

$$X^{(0)} = \begin{pmatrix} x_0 & x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 & x_7 \\ x_8 & x_9 & x_{10} & x_{11} \\ x_{12} & x_{13} & x_{14} & x_{15} \end{pmatrix}. \tag{11}$$

For a 256-bit key K, 32-bit block counter c, and 96-bit nonce N, the state is initialized as

$$X^{(0)} = \begin{pmatrix} \texttt{61707865} & \texttt{3320646e} & \texttt{79622d32} & \texttt{6b206574} \\ k_0 & k_1 & k_2 & k_3 \\ k_4 & k_5 & k_6 & k_7 \\ c & n_0 & n_1 & n_2 \end{pmatrix}. \tag{12}$$

The first row is the fixed constant corresponding to the little-endian encoding of "expand 32-byte k." The second and third rows contain the eight 32-bit words derived from the 256-bit key. The fourth row contains the block counter and three 32-bit words derived from the 96-bit nonce. This matrix formulation is useful because it makes the diffusion structure of ChaCha20 explicit: each round repeatedly mixes words across columns and diagonals until the output becomes computationally indistinguishable from random to an adversary without the key.

A.5 Quarter-Round Transformation

The elementary transformation in ChaCha20 is the *quarter round*, which updates four 32-bit words (a, b, c, d). All additions are performed modulo 2^{32}, $\oplus$ denotes bitwise XOR, and $\lll r$ denotes left rotation by r bits.

$$a \leftarrow a + b \pmod{2^{32}}, \tag{13}$$
$$d \leftarrow (d \oplus a) \lll 16, \tag{14}$$
$$c \leftarrow c + d \pmod{2^{32}}, \tag{15}$$
$$b \leftarrow (b \oplus c) \lll 12, \tag{16}$$
$$a \leftarrow a + b \pmod{2^{32}}, \tag{17}$$
$$d \leftarrow (d \oplus a) \lll 8, \tag{18}$$
$$c \leftarrow c + d \pmod{2^{32}}, \tag{19}$$
$$b \leftarrow (b \oplus c) \lll 7. \tag{20}$$

This transformation combines three operations: modular addition, XOR, and bit rotation. The interaction among these operations is central to ChaCha20's diffusion. Modular addition introduces carry-dependent nonlinearity, XOR mixes bit patterns, and rotations move bit positions so that local changes spread across words over repeated rounds.

The quarter round can be denoted compactly as

$$(a', b', c', d') = QR(a, b, c, d). \tag{21}$$

Because modular addition, XOR, and fixed rotation are each invertible on 32-bit words, the quarter-round operation is also structurally reversible. However, ChaCha20 is not used by explicitly inverting this transformation. Instead, the repeated transformations generate a keystream block, and encryption/decryption rely on XOR symmetry.

A.6 Column Rounds, Diagonal Rounds, and Double Rounds

One ChaCha20 double round contains one column round followed by one diagonal round. The column round applies the quarter-round function to the four columns:

$$QR(x_0, x_4, x_8, x_{12}), \tag{22}$$

$$QR(x_1, x_5, x_9, x_{13}), \tag{23}$$

$$QR(x_2, x_6, x_{10}, x_{14}), \tag{24}$$

$$QR(x_3, x_7, x_{11}, x_{15}). \tag{25}$$

The diagonal round then applies the quarter-round function to the four diagonals:

$$QR(x_0, x_5, x_{10}, x_{15}), \tag{26}$$

$$QR(x_1, x_6, x_{11}, x_{12}), \tag{27}$$

$$QR(x_2, x_7, x_8, x_{13}), \tag{28}$$

$$QR(x_3, x_4, x_9, x_{14}). \tag{29}$$

Let $D(\cdot)$ denote one full double round. ChaCha20 applies ten double rounds:

$$X^{(20)} = D^{10}\left(X^{(0)}\right). \tag{30}$$

After the twentieth round, the original state is added back word by word:

$$Y_i = X_i^{(20)} + X_i^{(0)} \pmod{2^{32}}, \qquad i = 0, \ldots, 15. \tag{31}$$

The final 64-byte block is obtained by serializing the sixteen words in little-endian order:

$$B(K, N, c) = \mathrm{LE}_{32}(Y_0) \parallel \mathrm{LE}_{32}(Y_1) \parallel \cdots \parallel \mathrm{LE}_{32}(Y_{15}). \tag{32}$$

The feed-forward addition of $X^{(0)}$ to $X^{(20)}$ ensures that the output block depends on both the heavily transformed state and the original key-counter-nonce structure. In the project setting, this output block becomes part of the keystream used to encrypt packet payloads.

A.7 ChaCha20 Encryption and Correctness Proof

For a plaintext packet P of length m bytes, ChaCha20 generates enough 64-byte blocks to cover the packet:

$$S = B(K, N, 1) \parallel B(K, N, 2) \parallel \cdots \parallel B(K, N, q), \tag{33}$$

where

$$q = \left\lceil \frac{m}{64} \right\rceil. \tag{34}$$

The keystream is truncated to the plaintext length:

$$S_m = S[0 : m]. \tag{35}$$

Encryption is performed by XOR:

$$C = P \oplus S_m. \tag{36}$$

Decryption applies the same keystream:

$$P' = C \oplus S_m. \tag{37}$$

Substituting the encryption equation gives

$$\begin{aligned} P' &= (P \oplus S_m) \oplus S_m && (38)\\ &= P \oplus (S_m \oplus S_m) && (39)\\ &= P \oplus 0 && (40)\\ &= P. && (41) \end{aligned}$$

Therefore, ChaCha20 decryption is correct. This proof establishes functional correctness. The security argument additionally requires that the keystream S_m be computationally indistinguishable from random to an adversary without knowledge of K, and that the same key-nonce pair is never reused.

A.8 Necessity of Nonce Uniqueness

Nonce uniqueness is a strict condition for stream-cipher security. Suppose the same key K and nonce N are used to encrypt two different plaintext packets P_1 and P_2. Then both packets use the same keystream S:

$$C_1 = P_1 \oplus S, \qquad C_2 = P_2 \oplus S. \tag{42}$$

An adversary who observes both ciphertexts can compute

$$\begin{aligned} C_1 \oplus C_2 &= (P_1 \oplus S) \oplus (P_2 \oplus S) && (43)\\ &= P_1 \oplus P_2. && (44) \end{aligned}$$

The keystream cancels. This does not necessarily reveal both plaintexts immediately, but it reveals a deterministic relationship between them and destroys the intended confidentiality guarantee. In ChaCha20–Poly1305, nonce reuse is even more serious because it also causes reuse of the Poly1305 one-time key. Therefore, the VPN implementation must assign a unique nonce to every packet under a fixed session key.

A practical nonce structure for the project can be defined as

$$N = \mathrm{LE}_{32}(\mathrm{sender_id}) \parallel \mathrm{LE}_{64}(\mathrm{packet_counter}). \tag{45}$$

This construction separates nonce space by sender and packet sequence. It also aligns with the system-level requirement that edge nodes transmit many packets over long-running encrypted channels without reusing encryption parameters.

A.9 Poly1305 One-Time Key Generation

Poly1305 requires a one-time key split into two 128-bit values:

$$\mathrm{OTK} = r \parallel s. \tag{46}$$

In ChaCha20–Poly1305, this one-time key is generated from the ChaCha20 block function using counter 0:

$$\mathrm{OTK} = B(K, N, 0)[0:32]. \tag{47}$$

The first 16 bytes form r, after clamping, and the next 16 bytes form s:

$$r = \mathrm{clamp}\left(\mathrm{LE}_{128}(\mathrm{OTK}[0:16])\right), \tag{48}$$

$$s = \mathrm{LE}_{128}(\mathrm{OTK}[16:32]). \tag{49}$$

The clamping operation restricts certain bits of r:

$$r \leftarrow r \;\&\; \texttt{0x0ffffffc0ffffffc0ffffffc0fffffff}. \tag{50}$$

This means that specific high and low bits are cleared. The purpose is not to make r secret; secrecy follows from the ChaCha20-derived one-time key. Rather, clamping constrains r to a form that supports efficient and safe arithmetic modulo $2^{130} - 5$.

A.10 Poly1305 Message Encoding

Poly1305 does not authenticate the plaintext directly in the AEAD construction. It authenticates a structured message containing associated data, ciphertext, padding, and length fields:

$$M = A \,\|\, \mathrm{pad}_{16}(A) \,\|\, C \,\|\, \mathrm{pad}_{16}(C) \,\|\, \mathrm{LE}_{64}(|A|) \,\|\, \mathrm{LE}_{64}(|C|). \tag{51}$$

The padding function is

$$\mathrm{pad}_{16}(X) = 0^{(16-|X| \bmod 16) \bmod 16}. \tag{52}$$

The length fields are necessary because they make the encoded authentication input unambiguous. Without them, different pairs of associated data and ciphertext could produce the same concatenated byte sequence after padding. By including $|A|$ and $|C|$, the receiver authenticates not only the bytes themselves but also their structural boundary.

In the project's VPN setting, this matters because packet metadata and ciphertext must be bound together. If an attacker changes the routing metadata, packet type, source identifier, or ciphertext, the reconstructed authentication input M changes, and the verification tag will fail.

A.11 Poly1305 Arithmetic over the Field $2^{130} - 5$

Let the encoded message M be divided into 16-byte blocks:

$$M = M_1 \,\|\, M_2 \,\|\, \cdots \,\|\, M_q. \tag{53}$$

Each block is interpreted as a little-endian integer with an appended high bit:

$$n_i = \mathrm{LE}(M_i \,\|\, \texttt{01}). \tag{54}$$

For a full 16-byte block, this is equivalent to

$$n_i = \mathrm{LE}(M_i) + 2^{128}. \tag{55}$$

Poly1305 operates modulo the prime

$$p = 2^{130} - 5. \tag{56}$$

The accumulator is initialized as

$$a_0 = 0. \tag{57}$$

For each block, the recurrence is

$$a_i = (a_{i-1} + n_i)r \pmod{p}, \qquad i = 1, \ldots, q. \tag{58}$$

After all blocks have been processed, the authentication tag is

$$T = \mathrm{LE}_{128}\left((a_q + s) \bmod 2^{128}\right). \tag{59}$$

The modulus $2^{130}-5$ gives Poly1305 its polynomial-field structure. The final reduction modulo 2^{128} reflects the fact that the transmitted tag is 128 bits.

A.12 Polynomial Expansion and Integrity Proof Sketch

The recurrence above explains why the mechanism is called Poly1305. Expanding the accumulator gives a polynomial in the secret multiplier r. For the first block,

$$a_1 = n_1 r \pmod{p}. \tag{60}$$

For the second block,

$$\begin{aligned} a_2 &= (a_1 + n_2)r && (61)\\ &\equiv (n_1 r + n_2)r && (62)\\ &\equiv n_1 r^2 + n_2 r \pmod{p}. && (63) \end{aligned}$$

For the third block,

$$\begin{aligned} a_3 &= (a_2 + n_3)r && (64)\\ &\equiv n_1 r^3 + n_2 r^2 + n_3 r \pmod{p}. && (65) \end{aligned}$$

By induction, for q blocks,

$$a_q \equiv \sum_{i=1}^{q} n_i r^{q-i+1} \pmod{p}. \tag{66}$$

Thus the authentication tag can be written as

$$T = \left(\sum_{i=1}^{q} n_i r^{q-i+1} + s\right) \bmod 2^{128}. \tag{67}$$

This expression shows that Poly1305 evaluates a polynomial whose coefficients are derived from the encoded message blocks. If an adversary modifies either the associated data or the ciphertext, then at least one block coefficient n_i changes. A successful forgery would require producing a different message M' and tag T' such that

$$T' = \mathrm{Poly1305}_{r,s}(M'). \tag{68}$$

When r and s are unknown and used only once, the adversary cannot efficiently compute the correct tag for the modified message. More formally, if two distinct messages $M \neq M'$ produce the same tag under the same one-time key, then the difference between their polynomial

evaluations yields a nonzero polynomial equation in r:

$$\sum_{i=1}^{q} n_i r^{q-i+1} - \sum_{i=1}^{q'} n'_i r^{q'-i+1} \equiv 0 \pmod{p}. \tag{69}$$

A nonzero polynomial over a field has a limited number of roots relative to its degree. Therefore, when r is unpredictable and not reused across messages, the probability of constructing a valid forgery remains bounded. This is the mathematical reason Poly1305 must be used with a one-time key.

This proof is a design-level proof sketch rather than a full cryptographic reduction. Its purpose in this report is to justify the engineering rule that the pair (r, s) must never be reused. Since (r, s) is derived from (K, N), this requirement reduces in implementation to enforcing nonce uniqueness under each session key.

A.13 Authentication Verification and Decryption Procedure

When the receiver obtains (C, T), it first reconstructs the authentication input:

$$M = A \parallel \mathrm{pad}_{16}(A) \parallel C \parallel \mathrm{pad}_{16}(C) \parallel \mathrm{LE}_{64}(|A|) \parallel \mathrm{LE}_{64}(|C|). \tag{70}$$

It then computes

$$T' = \mathrm{Poly1305}_{r,s}(M). \tag{71}$$

The packet is accepted only if

$$T' = T. \tag{72}$$

If the tag comparison fails, the packet is rejected:

$$\mathrm{AEAD.\,Dec}_{K,N}(C, A, T) = \perp. \tag{73}$$

If the tag comparison succeeds, the receiver regenerates the ChaCha20 keystream and decrypts:

$$P = C \oplus S_m. \tag{74}$$

The order of operations is significant. Authentication must precede trust. A packet should not be passed to Kafka, inserted into a retry spool as valid data, or written into the Bronze data lake unless its tag has been verified. This ordering protects the integrity of all downstream database layers.

A.14 Integrated AEAD Security Logic

The combined construction can be interpreted as the composition of two security mechanisms:

$$\mathrm{ChaCha20} \Rightarrow \text{confidentiality}, \tag{75}$$

$$\mathrm{Poly1305} \Rightarrow \text{integrity and authenticity}. \tag{76}$$

Together, they provide a dependency relationship:

$$\mathrm{AEADSecurity} = f\left(\mathrm{PseudorandomKeystream}, \mathrm{OneTimeAuthentication}, \mathrm{NonceUniqueness}\right). \tag{77}$$

This expression is not meant as a numerical equation. It represents a dependency structure. ChaCha20–Poly1305 is secure only when the keystream is computationally pseudorandom, the Poly1305 key is one-time, and nonce uniqueness is enforced. If nonce uniqueness fails, the same keystream may be reused and the same Poly1305 key may be regenerated. In that case, both confidentiality and integrity assumptions become invalid.

For the proposed trading platform, this security logic has direct architectural implications. Since edge collectors continuously transmit data packets, each sender must maintain a monotonic packet counter. A session must be rekeyed before counter exhaustion. Bidirectional communication must use separate nonce spaces or direction-specific keys. These requirements are not implementation details; they are conditions under which the mathematical security argument remains valid.

A.15 Diagrammatic Representation of the Encryption Pipeline

A.16 Implications for the Project's Data Architecture

The encryption mechanism supports the database architecture by protecting records before they enter the storage system. Let R be a data record extracted from a decrypted packet. The platform should admit R into Kafka only if

$$\mathrm{Verify}_{K,N}(A, C, T) = 1. \tag{78}$$

After admission, the database layer computes a payload hash:

$$h(R) = H(\mathrm{canon}(R)), \tag{79}$$

where $\mathrm{canon}(R)$ is the deterministic serialized representation of the record. The first condition protects the packet during transmission; the second protects record lineage after ingestion. The two mechanisms therefore operate at different layers but support the same integrity objective.

This layered protection is consistent with the broader system architecture. The VPN channel protects edge-to-center transfer, Kafka buffers validated messages, the Bronze layer stores immutable raw evidence, the Silver layer normalizes validated records, and the Gold layer constructs model-facing features and logs. If packet-level authentication were absent, corrupted records could propagate through all layers and compromise downstream trading decisions. Therefore, ChaCha20–Poly1305 is not an auxiliary security component but a prerequisite for reliable database construction.

A.17 Limitations and Boundary Conditions

The correctness and security arguments above depend on several implementation assumptions. First, session keys must remain secret. If the key K is exposed, both confidentiality and authentication are compromised. Second, nonces must be unique under each session key. Nonce reuse invalidates both stream-cipher confidentiality and Poly1305 one-time authentication. Third, tag verification must be performed before decrypted data are trusted. Fourth, tag comparison should be implemented in constant time to avoid timing leakage. Fifth, the design

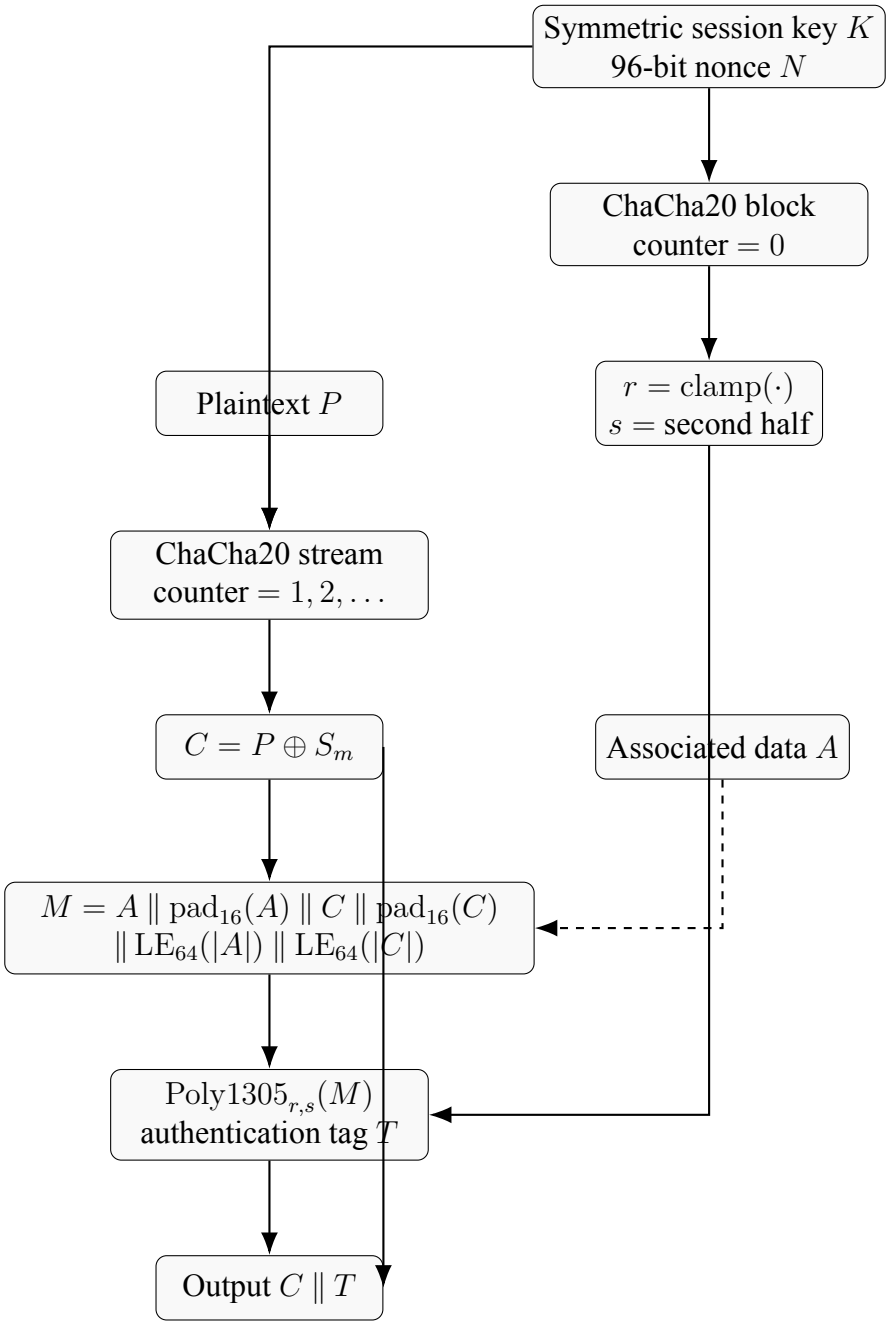


Figure 16: ChaCha20–Poly1305 authenticated encryption pipeline. ChaCha20 with counter 0 derives the one-time Poly1305 key, while ChaCha20 with counter 1 and later counters encrypts the plaintext. Poly1305 then authenticates the associated data and ciphertext, so confidentiality and integrity are jointly enforced before data enter the central pipeline.

protects data in transit, but it does not by itself guarantee database correctness after ingestion; post-ingestion integrity still requires hashing, schema validation, access control, and audit logs.

The analysis also assumes that ChaCha20 behaves as a secure pseudorandom keystream generator and that Poly1305 is used as a one-time polynomial authenticator. These are standard cryptographic assumptions for the construction. The appendix does not attempt to prove these primitives from first principles; instead, it explains how their mathematical structure supports the system-level security requirements when implemented correctly.

A.18 Concluding Transition

ChaCha20–Poly1305 provides the mathematical basis for secure packet transmission in the proposed fintech platform. ChaCha20 encrypts packet payloads by generating a pseudorandom keystream from a session key, nonce, and counter. Poly1305 authenticates associated data and ciphertext through polynomial evaluation over the field defined by $2^{130} - 5$. The combined AEAD construction ensures that only packets satisfying both confidentiality and integrity conditions are admitted into the data pipeline.

This appendix therefore complements the database design by showing how transmission security supports data reliability. The Bronze–Silver–Gold architecture can preserve and transform data correctly only if incoming records are authentic and unmodified. ChaCha20–Poly1305 supplies this protection at the network boundary, while database hashes, timestamps, and lineage fields maintain integrity after ingestion.